p-adic GRN Suite

Plataforma integral para análisis p-ádico de redes de regulación genética

Jesús Rogelio Pérez-Buendía Víctor Nopal-Coello Yuriria Cortés-Poza

Periodo de desarrollo: 2023–2026

Resumen

Este documento presenta el software científico *p-adic GRN Suite*, diseñado para el análisis p-ádico de redes de regulación genética en dinámica discreta finita. El software está organizado en tres partes funcionales: *core* (métricas p-ádicas), *search* (búsqueda/certificación de orderings) y *viz* (visualización y reportes). El reporte reúne documentación funcional, manuales de uso, scripts de validación, código fuente, pruebas automatizadas, ejemplos de corridas con resultados reales y evidencia reproducible, además del estado actual del demo web institucional.

Índice

1 Introducción y alcance del software **3**

2 Contribución científica verificable **3**

3 Ventajas, necesidad y valor del software **4**

4 Alcance operativo y condiciones de uso **5**

5 Datos generales del software **5**
5.1 Enlaces oficiales . . . 5
5.2 Actualización de enlaces . . . 6

6 Manual de usuario de la suite **6**

7 Ejemplos de corridas de la suite **8**
7.1 Resumen claro de corridas con resultados reales . . . 8
7.2 Documento detallado de ejemplos . . . 9
7.3 Correos electrónicos oficiales de autores . . . 10
7.4 Financiamiento . . . 11
7.5 Autoría oficial consolidada . . . 11

8 Sección I: Módulo core (métricas p-ádicas) **11**
8.1 README . . . 11
8.2 Manual de usuario . . . 12
8.3 Manual técnico . . . 13
8.4 Configuración de paquete . . . 15
8.5 Cita académica . . . 15

8.6 Licencia . . . 16
8.7 Configuración de pruebas . . . 16
8.8 Código fuente . . . 16
8.9 Pruebas y ejemplos . . . 23
8.10 Evidencia de validación . . . 23

9 Sección II: Módulo search (búsqueda/certificación) 25
9.1 Programas de optimización incluidos . . . 25
9.2 README . . . 25
9.3 Manual de usuario . . . 26
9.4 Manual técnico . . . 27
9.5 Configuración de paquete . . . 28
9.6 Cita académica . . . 28
9.7 Licencia . . . 29
9.8 Configuración de pruebas . . . 29
9.9 Código fuente . . . 29
9.10 Pruebas . . . 37
9.11 Evidencia de validación . . . 38

10 Sección III: Módulo viz (visualización/reportes) 39
10.1 README . . . 39
10.2 Manual de usuario . . . 40
10.3 Manual técnico . . . 41
10.4 Configuración de paquete . . . 42
10.5 Cita académica . . . 42
10.6 Licencia . . . 43
10.7 Configuración de pruebas . . . 43
10.8 Código fuente . . . 43
10.9 Pruebas . . . 45
10.10 Evidencia de validación . . . 46

11 Scripts globales y corridas de integración 47
11.1 Script maestro de validación . . . 47
11.2 Evidencia de casos de estudio . . . 47

12 Applets y UI web reportada 53
12.1 Estado actual . . . 53
12.2 Alcance operativo del demo web . . . 53
12.3 README del demo web . . . 53
12.4 Frontend HTML . . . 53
12.5 Backend . . . 54
12.6 Guía de despliegue . . . 55
12.7 Capturas de pantalla del demo . . . 55

1. Introducción y alcance del software

p-adic GRN Suite es un software amplio y especializado para análisis p-ádico de redes de regulación genética. Su diseño es general para sistemas de dinámica discreta finita y su operación integra cálculo, búsqueda/certificación y visualización en un flujo científico reproducible. Este software consta de tres partes técnicas que se usan de forma coordinada en un mismo flujo de trabajo:

- **core**: cálculo de métricas p-ádicas sobre dinámicas discretas finitas.
- **search**: búsqueda/certificación de orderings con estrategias computacionales.
- **viz**: visualización y comunicación reproducible de resultados.

Relación operacional de uso:

1. Se define el sistema y se calculan métricas con *core*.
2. Se exploran/certifican orderings con *search* usando el mismo marco matemático.
3. Se generan figuras y salidas de reporte con *viz*.

Con este esquema, la suite mantiene coherencia científica y técnica, y documenta en un solo expediente todo el proceso de análisis.

Fundamentos teóricos y referencias

Los fundamentos matemáticos del método están desarrollados en las siguientes referencias. Sobre esta base teórica, el software se reporta como plataforma de análisis computacional para redes regulatorias en dinámica discreta finita:

- Pérez-Buendía, J. R.; Nopal-Coello, V. *Hierarchical p-Adic Framework for Gene Regulatory Networks: Theory and Stability Analysis*. arXiv:2603.14097. Disponible en: `https://arxiv.org/abs/2603.14097`
- Pérez-Buendía, J. R.; Nopal-Coello, V.; Cortés-Poza, Y. *Regulatory p-Adic Orderings from Stability in Gene Networks: Applications and Validation in Three Developmental Systems*. Zenodo, registro 19482987. Disponible en: `https://zenodo.org/records/19482987`

2. Contribución científica verificable

La *p-adic GRN Suite* implementa de manera reproducible una metodología computacional para el análisis matemático de redes de regulación genética en dinámica discreta finita, articulando en un solo flujo tres capacidades complementarias: cálculo de métricas p-ádicas, búsqueda/certificación de orderings y visualización de resultados.

Funcionalidad que habilita

La suite organiza en un mismo entorno de trabajo tres actividades centrales del análisis p-ádico de dinámicas discretas: (i) cálculo de indicadores, (ii) exploración de estructuras de ordenamiento y (iii) generación de salidas interpretables para discusión científica. Esta integración fortalece la coherencia metodológica y la reproducibilidad del flujo completo.

Novedad y aporte

El aporte verificable de software consiste en operacionalizar, dentro de una arquitectura modular y testeada, métodos p-ádicos de análisis de redes en dinámica discreta finita para:

- cuantificar comportamiento p-ádico por medio de `mu_E`, `mu_A`, `mu_I` y sus perfiles asociados;
- identificar orderings candidatos y certificarlos con estrategias computacionales (heurísticas y exhaustivas);
- generar artefactos de comunicación reproducibles para validación y reporte.

Evidencia reproducible incluida

La contribución está respaldada por:

- código fuente ejecutable y versionado en las tres partes del software;
- pruebas automatizadas y corridas de validación en cada módulo;
- ejemplos de corridas con salidas numéricas reales incluidas en este expediente;
- script maestro de validación integral para ejecutar la suite de forma unificada.

3. Ventajas, necesidad y valor del software

Necesidad científica y técnica

El análisis de redes de regulación genética con enfoque p-ádico requiere integrar, en un mismo flujo, cálculo matemático, exploración sistemática de orderings y visualización interpretable de resultados. La suite proporciona ese flujo integrado con estructura operativa y trazabilidad reproducible.

Ventajas principales

- Integra en un solo entorno operativo las tres etapas del análisis: *core*, *search* y *viz*.
- Estandariza entradas, salidas y validaciones, fortaleciendo la consistencia de ejecución y la auditoría técnica.
- Permite combinar búsqueda heurística y certificación sistemática para estudiar orderings con criterio cuantitativo.
- Produce resultados verificables y comunicables mediante reportes, figuras y archivos de evidencia.

Aporte y valor para la comunidad

El software aporta una infraestructura reproducible para llevar métodos p-ádicos del plano teórico al análisis computacional aplicado. Su valor reside en hacer operativo un flujo completo de trabajo, con trazabilidad de resultados y documentación utilizable por personas investigadoras, estudiantes y equipos técnicos en biología matemática y sistemas dinámicos discretos.

Uso reportado

En este expediente se documenta su uso mediante corridas reproducibles, pruebas automatizadas y validaciones integradas para las tres partes de la suite. Asimismo, las referencias citadas en la introducción reportan su aplicación en redes regulatorias de *Arabidopsis thaliana*, *Caenorhabditis elegans* y *Drosophila melanogaster*. Su uso también se reporta en líneas de investigación activas para problemas particulares de regulación genética y otros sistemas biológicos analizados por equipos de investigación.

4. Alcance operativo y condiciones de uso

Alcance operativo

La versión documentada corresponde a una suite científica funcional para análisis p-ádico de redes de regulación genética en dinámica discreta finita, con resultados reproducibles en línea de comandos, visualizaciones y evidencia formal en pruebas y archivos de salida.

Condiciones de uso reportadas

- El software incluye tres partes coordinadas: *core*, *search* y *viz*, integradas en un flujo único de trabajo.
- El demo web institucional proporciona interfaz de consulta, backend de verificación y guía de despliegue para uso demostrativo.
- Este expediente incorpora documentación técnica, manuales de usuario, ejemplos de corridas con resultados reales y scripts de validación.
- Los campos de DOI y enlaces oficiales se reportan con identificadores de referencia y se actualizan con los identificadores definitivos de publicación.

5. Datos generales del software

- Nombre de software: `p-adic GRN Suite`
- Versión de integración documental: `1.0.0`
- Lenguaje principal: Python 3.10+
- Partes del software: `padic-grn-core`, `padic-grn-search`, `padic-grn-viz`
- DOI (Zenodo): `DOI_ZENODO_SUITE`

5.1. Enlaces oficiales

- Repositorio GitHub (suite): `URL_GITHUB_SUITE`
- Release GitHub (versión actual): `URL_GITHUB_RELEASE`
- Registro Zenodo (suite): `URL_ZENODO_SUITE`
- DOI de versión (snapshot Zenodo): `DOI_VERSION`
- DOI conceptual (todas las versiones): `DOI_CONCEPTUAL`

5.2. Actualización de enlaces

Los identificadores de referencia se reemplazan con enlaces y DOI oficiales una vez emitidos por los repositorios institucionales correspondientes.

6. Manual de usuario de la suite

````
# Manual de usuario — p-adic GRN Suite

## 1. Qué es p-adic GRN Suite

`p-adic GRN Suite` integra tres módulos:

- `padic-grn-core`: cálculo de métricas p-ádicas (`mu_E`, `mu_A`, `mu_I`).
- `padic-grn-search`: búsqueda/certificación de orderings.
- `padic-grn-viz`: visualización y reporte gráfico.

Flujo recomendado de uso:

1. calcular métricas con `core`,
2. explorar/certificar orderings con `search`,
3. visualizar y documentar resultados con `viz`.

## 2. Requisitos

- Python 3.10+
- `pip`
- entorno local con acceso a los tres submódulos en `software-suite/`

## 3. Instalación (modo desarrollo)

```bash
cd software-suite/padic-grn-core && pip install -e .
cd ../padic-grn-search && pip install -e .
cd ../padic-grn-viz && pip install -e .
```

## 4. Entrada mínima esperada

Para `core/search` se usa un CSV de transiciones discretas con columnas:

- `config_from`
- `config_to`

Para `viz` se usan resultados JSON/CSV producidos por las corridas anteriores.

## 5. Uso rápido de la suite

### 5.1 Calcular un ordering con core

```bash
cd software-suite/padic-grn-core
pgcore-score-ordering examples/transitions_p2_n2_identity.csv 2 2 "0 1"
↪  --output-json core_scores.json
````
````

```

Salida esperada: archivo `core_scores.json`.

### 5.2 Búsqueda con search

```bash
cd ../padic-grn-search
pgsearch-ga ../padic-grn-core/examples/transitions_p2_n2_identity.csv 2 2
↪  --objective minimize_mu_E --generations 5 --population 10
```

Salida esperada: mejor ordering y métricas objetivo.

5.3 Visualización con viz

```bash
cd ../padic-grn-viz
pgviz-dependency-graph
pgviz-hierarchical-graph
```

Salida esperada: archivos de figura/reportes (según parámetros de ejecución).

6. Ejecución de validación integrada

```bash
cd software-suite
./RUN-ALL-VALIDATIONS.sh
```

Salida esperada:

- pruebas `pytest` para `core`, `search`, `viz`,
- estado global de validación de la suite.

7. Buenas prácticas

- guardar junto al resultado: parámetros, versión y fecha de corrida,
- mantener nombres de archivos estables para reproducibilidad,
- usar siempre ejemplos pequeños antes de correr casos grandes.

8. Solución de problemas

- **Comando no encontrado**: verificar instalación editable (`pip install -e
↪ .`) en cada módulo.
- **CSV inválido**: revisar columnas y formato de tabla de transiciones.
- **Ordering inválido**: verificar que sea una permutación sin repetición de
↪ `0..N-1`.
- **Error de visualización**: confirmar instalación de dependencias gráficas.
```

7. Ejemplos de corridas de la suite

7.1. Resumen claro de corridas con resultados reales

Esta subsección sintetiza corridas reales verificables para lectura rápida del evaluador, incluyendo entrada, comando, salida principal e interpretación.

Corrida 1: scoring con *core* (toy, $p = 2$, $N = 2$)

- Entrada: `padic-grn-core/examples/transitions_p2_n2_identity.csv`
- Comando: `pgcore-score-ordering ... 2 2 "0 1-output-json`
- Evidencia de salida: `core_scores_p2.json` (carpeta de evidencia de corridas)
- Resultado real: `mu_E=0`, `mu_A=0`, `mu_I=4`
- Interpretación: en este caso de referencia, el comportamiento reportado es de indiferencia p-ádica predominante (`mu_I` máximo relativo).

Corrida 2: búsqueda genética con *search*

- Objetivo: `minimize_mu_E`
- Evidencia de salida: `search_ga_p2.json` (carpeta de evidencia de corridas)
- Resultado real: método GA con PMX, `best_ordering=[1,0]`, `best_fitness=0`, `evaluations=2`
- Resultado real de métricas: `mu_E=0`, `mu_A=0`, `mu_I=4`

Corrida 3: certificación exacta con *search*

- Evidencias: `search_bbmin_p2.json` y `search_bbmax_p2.json`
- Métodos reales reportados: branch-and-bound con cota parcial para minimización de `mu_E` y backtracking exacto para maximización de `mu_A`.
- Resultado real en el caso toy: `best_ordering=[0,1]` y `best_fitness=0` para ambos objetivos.

Corrida 4: validación global de suite

- Script maestro: `RUN-ALL-VALIDATIONS.sh`
- Alcance: ejecución de validaciones de *core*, *search* y *viz* en una sola corrida.
- Evidencia complementaria: archivos `validacion_*_pytest.txt` en cada módulo.

7.2. Documento detallado de ejemplos

````
# Ejemplos de corridas — p-adic GRN Suite

Este documento resume corridas representativas para la suite completa.

## Ejemplo 1: pipeline mínimo (toy p=2, N=2) con salida real

### Paso A. Scoring de ordering con core

```bash
cd software-suite/padic-grn-core
pgcore-score-ordering examples/transitions_p2_n2_identity.csv 2 2 "0 1"
↪  --output-json toy_core_scores.json
```

Archivo de salida:

- `toy_core_scores.json`

Salida real de referencia (`core_scores_p2.json`):

```json
{
  "mu_E": 0,
  "mu_A": 0,
  "mu_I": 4
}
```

### Paso B. Búsqueda con search

```bash
cd ../padic-grn-search
pgsearch-ga ../padic-grn-core/examples/transitions_p2_n2_identity.csv 2 2
↪  --objective minimize_mu_E --generations 5 --population 10
```

Resultado real de referencia (`search_ga_p2.json`):

- `method`: `ga_pmx_tournament_elite`
- `best_ordering`: `[1, 0]`
- `best_fitness`: `0`
- `best_scores`: `mu_E=0`, `mu_A=0`, `mu_I=4`
- `evaluations`: `2`

### Paso C. Visualización con viz

```bash
cd ../padic-grn-viz
pgviz-dependency-graph
pgviz-hierarchical-graph
```

Resultado real actual:
````

```
- el módulo `viz` está funcional para generación de gráficas,
- en esta versión las capturas institucionales se reportan en:
  - `web-demo-institucional/screenshots/demo-ui-desktop.png`
  - `web-demo-institucional/screenshots/demo-ui-mobile.png`

## Ejemplo 2: validación completa de suite

```bash
cd software-suite
./RUN-ALL-VALIDATIONS.sh
```

Resultado real (última corrida reportada):

- ejecución de pruebas en los tres módulos,
- confirmación de estado global.

Ejemplo 3: búsqueda exhaustiva (valores reales)

Resultados de referencia:

- `search_bbmin_p2.json`
  - `method`: `branch_and_bound_partial_muE`
  - `objective`: `minimize_mu_E`
  - `best_ordering`: `[0, 1]`
  - `best_fitness`: `0`
  - `best_scores`: `mu_E=0`, `mu_A=0`, `mu_I=4`
- `search_bbmax_p2.json`
  - `method`: `backtracking_exact`
  - `objective`: `maximize_mu_A`
  - `best_ordering`: `[0, 1]`
  - `best_fitness`: `0`
  - `best_scores`: `mu_E=0`, `mu_A=0`, `mu_I=4`

Ejemplo 4: evidencias de referencia incluidas

Para revisión documental se incluyen salidas de referencia en:

- `registro-dossier/06_evidencia_corridas/core_scores_p2.json`
- `registro-dossier/06_evidencia_corridas/core_scores_p3.json`
- `registro-dossier/06_evidencia_corridas/search_ga_p2.json`
- `registro-dossier/06_evidencia_corridas/search_bbmin_p2.json`
- `registro-dossier/06_evidencia_corridas/search_bbmax_p2.json`

Estas salidas permiten verificar reproducibilidad contra corridas previas y se
usan como base documental en el PDF principal de la suite.
```

7.3. Correos electrónicos oficiales de autores

- Jesús Rogelio Pérez-Buendía: rogelio.perez@cimat.mx
- Víctor Nopal-Coello: victor_nopal@uaeh.edu.mx
- Yuriria Cortés-Poza: yuriria.cortes@iimas.unam.mx

7.4. Financiamiento

- Modelos matemáticos y computacionales no convencionales para el estudio y análisis de problemas relevantes en Biología, CF 2019/217367.

7.5. Autoría oficial consolidada

La autoría, ORCID y correos institucionales presentados en este expediente corresponden a la versión oficial del software y se reportan de forma consistente en portada, metadatos de paquete y archivos de citación.

8. Sección I: Módulo core (métricas p-ádicas)

8.1. README

````
# padic-grn-core: Núcleo p-ádico para sistemas dinámicos discretos

## Resumen

Paquete base para calcular métricas p-ádicas (`mu_E`, `mu_A`, `mu_I`) en
↪  sistemas dinámicos discretos finitos.

## Introducción

`padic-grn-core` concentra el cálculo matemático central de la suite. Está
↪  orientado a redes de regulación genética multivaluadas y, mediante
↪  adaptadores, puede emplearse también en sistemas discretos generales.

## Comandos

- `pgcore-score-ordering`
- `pgcore-batch-scores`

## Uso rápido

```bash
pgcore-score-ordering transitions.csv 13 2 "12 4 3 8 1 5 7 2 10 11 9 6 0"
↪  --output-json scores.json
```

`pgcore-batch-scores` espera un CSV con columna `ordering`.

## Alcance objetivo

- Soporte oficial: GRN deterministas multivaluadas.
- Soporte experimental: sistemas discretos generales via adaptadores.

## Autoría oficial

1. Jesús Rogelio Pérez-Buendía (SECIHTI-CIMAT, Unidad Mérida) — ORCID
↪  `0000-0002-7739-4779`
2. Víctor Nopal-Coello (Universidad Autónoma del Estado de Hidalgo, UAEH) —
↪  ORCID `0000-0003-2608-3636`
````

```
3. Yuriria Cortés-Poza (IIMAS, UNAM – Unidad Académica Yucatán) – ORCID
↪  `0000-0003-1856-9206`

## DOI

- DOI del software: `DOI_SOFTWARE_CORE`.

## Financiamiento

Modelos matemáticos y computacionales no convencionales para el estudio y
↪  análisis de problemas relevantes en Biología, CF 2019/217367.
```

8.2. Manual de usuario

```
# Manual de usuario – padic-grn-core

## 1. Requisitos

- Python 3.10+
- Archivo CSV de transiciones

## 2. Instalación local (modo desarrollo)

```bash
cd software-suite/padic-grn-core
pip install -e .
```

3. Uso básico: un ordenamiento

```bash
pgcore-score-ordering transitions.csv 13 2 "12 4 3 8 1 5 7 2 10 11 9 6 0"
↪  --output-json scores.json
```

Resultado:

- genera `scores.json` con `mu_E`, `mu_A`, `mu_I`.

4. Uso por lote (batch)

`orderings.csv` debe tener una columna `ordering`.

```bash
pgcore-batch-scores transitions.csv 13 2 orderings.csv --output-csv
↪  batch_scores.csv
```

Resultado:

- `batch_scores.csv` con columnas:
  - `ordering`
  - `mu_E`
  - `mu_A`
  - `mu_I`
```

```
## 5. Buenas prácticas

- usar rutas relativas limpias y evitar espacios en nombres de archivos de
↪  trabajo.
- guardar siempre los parámetros de corrida junto al resultado.
- versionar los archivos de entrada críticos.

## 6. Solución de problemas

- Error “ordering inválido”:
  - verifica que contenga todos los índices `0..N-1` sin repetición.
- Error de CSV:
  - verifica columnas `config_from,config_to`.
- Error de módulo legado no encontrado:
  - ejecutar desde el workspace donde existe el árbol fuente oficial.
```

8.3. Manual técnico

```
# Manual técnico — padic-grn-core

## 1. Propósito científico

`padic-grn-core` implementa el núcleo matemático para cuantificar estabilidad
↪  jerárquica en sistemas dinámicos discretos finitos mediante métricas
↪  p-ádicas:

- `mu_E` (expansión),
- `mu_A` (contracción),
- `mu_I` (isometría/indiferencia),
- perfiles por nivel `E(n), A(n), I(n)`.

Este módulo es la base de `padic-grn-search` y `padic-grn-viz`.

## 2. Modelo matemático formal

Sea `f: C -> C` un sistema discreto determinista sobre `|C| = p^N`.

1. Se fija un ordenamiento `pi` de variables.
2. Se identifica cada estado con su expansión base `p`.
3. Para cada nivel `n = 1, ..., N-1`, el espacio se particiona en
↪  cilindros/bolas `B_{n,m}`.
4. Para cada bola, se calcula `M_{n,m}` = primer nivel de discrepancia en
↪  imágenes bajo `f`.
5. Se clasifica:
   - `E` si `M_{n,m} < n`,
   - `I` si `M_{n,m} = n`,
   - `A` si `M_{n,m} > n`.
6. Con peso `w_n = p^(N-n)`:
   - `mu_E = sum_n E(n) * w_n`
   - `mu_I = sum_n I(n) * w_n`
   - `mu_A = sum_n A(n) * w_n`

Invariante de consistencia por nivel:
```

- `E(n) + I(n) + A(n) = p^n`.

3. Arquitectura de implementación

- `src/padic_grn_core/cli.py`
  - `pgcore-score-ordering`
  - `pgcore-batch-scores`
- `src/padic_grn_core/adapters.py`
  - `from_grn_table(...)` (estable)
  - `from_discrete_system(...)` (interfaz extendida)

Motor de cálculo:

- La CLI se conecta al módulo validado de referencia:
  - `20250115-p-adic-GRN-rewrite/articulo-final/supplementary/code/core/compute_mu_generalized.py`

4. Contrato de datos

Entrada mínima:

- `N` (entero positivo),
- `p` (entero >= 2),
- `transition_table.csv` con columnas:
  - `config_from`
  - `config_to`,
- `ordering` como permutación de `0..N-1`.

Salida estándar (`scores.json`):

- metadatos (`software`, `version`, `N`, `p`, `ordering`),
- `scores.mu_E`, `scores.mu_A`, `scores.mu_I`,
- perfiles por nivel.

5. Complejidad y límites prácticos

- Cálculo de una permutación: depende de `sum_{n=1}^{N-1} p^n` y del costo por bola.
- Para `N` grande, el cuello de botella está en evaluación masiva de bolas.
- El módulo soporta paralelismo en backend legado; la capa empaquetada lo expone indirectamente.

6. Validación y trazabilidad

Validaciones obligatorias:

- ordering válido (permutación),
- CSV con cabeceras correctas,
- integridad de tabla de transición.

Trazabilidad:

- toda corrida debe guardar: parámetros, versión, ruta de datos y hash de entrada.

```
## 7. Extensibilidad técnica

1. Consolidar el motor en `src/padic_grn_core/engine.py`.
2. Añadir hash de entrada y checksum de salida.
3. Añadir pruebas deterministas para `p=2` y `p=3` en CI.
4. Añadir API python estable:
   - `compute_scores(...)`,
   - `validate_transition_table(...)`.
```

8.4. Configuración de paquete

```
[build-system]
requires = ["setuptools>=68", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "padic-grn-core"
version = "0.1.0"
description = "Core general para metricas p-adicas en sistemas discretos."
readme = "README.md"
requires-python = ">=3.10"
license = {text = "Licencia propietaria para uso academico no comercial"}
authors = [
  {name = "Jesús Rogelio Pérez-Buendía", email = "rogelio.perez@cimat.mx"},
  {name = "Víctor Nopal-Coello", email = "victor_nopal@uaeh.edu.mx"},
  {name = "Yuriria Cortés-Poza", email = "yuriria.cortes@iimas.unam.mx"}
]
dependencies = []

[project.scripts]
pgcore-score-ordering = "padic_grn_core.cli:main_score_ordering"
pgcore-batch-scores = "padic_grn_core.cli:main_batch_scores"

[tool.setuptools]
package-dir = {"" = "src"}

[tool.setuptools.packages.find]
where = ["src"]
```

8.5. Cita académica

```
cff-version: 1.2.0
message: "If you use this software, please cite it as below."
title: "padic-grn-core"
version: "0.1.0"
type: software
authors:
  - family-names: "Pérez-Buendía"
    given-names: "Jesús Rogelio"
    affiliation: "SECIHTI - Centro de Investigación en Matemáticas (CIMAT),
    ↪  Unidad Mérida"
    orcid: "https://orcid.org/0000-0002-7739-4779"
  - family-names: "Nopal-Coello"
    given-names: "Víctor"
    affiliation: "Universidad Autónoma del Estado de Hidalgo (UAEH)"
```

```
    orcid: "https://orcid.org/0000-0003-2608-3636"
  - family-names: "Cortés-Poza"
    given-names: "Yuriria"
    affiliation: "Instituto de Investigaciones en Matemáticas Aplicadas y en
    ↪  Sistemas (IIMAS), UNAM -- Unidad Académica Yucatán"
    orcid: "https://orcid.org/0000-0003-1856-9206"
abstract: "Core general para cálculo de métricas p-ádicas (mu_E, mu_A, mu_I) y
↪  análisis base en sistemas dinámicos discretos."
license: "Licencia propietaria para uso academico no comercial"
repository-code: "PENDING_REPOSITORY_URL"
doi: "10.0000/pending-doi"
```

8.6. Licencia

```
Licencia propietaria para uso académico no comercial
Copyright (c) 2026
Jesús Rogelio Pérez-Buendía, Víctor Nopal-Coello, Yuriria Cortés-Poza

Permisos:
- Se autoriza el uso académico y de investigación sin fines de lucro.
- Se autoriza la evaluación técnica y científica del software.

Restricciones:
- No se autoriza la redistribución de copias sin permiso escrito.
- No se autoriza sublicenciar el software sin permiso escrito.
- No se autoriza el uso comercial sin permiso escrito.
- No se autoriza modificar y publicar versiones derivadas sin permiso escrito.

Contacto para autorizaciones:
- Jesús Rogelio Pérez-Buendía: rogelio.perez@cimat.mx

Esta licencia aplica a la versión actual del software y a los materiales
incluidos en este repositorio, salvo que un archivo específico indique
explícitamente otra condición.
```

8.7. Configuración de pruebas

```
[pytest]
pythonpath = src
```

8.8. Código fuente

```
"""padic-grn-core package."""

__all__ = ["__version__"]
__version__ = "0.1.0"

"""CLI funcional para padic-grn-core."""

from __future__ import annotations

import argparse
import csv
import importlib.util
import json
```

```
import sys
from pathlib import Path
from typing import Any


def _workspace_root() -> Path:
    current = Path(__file__).resolve()
    for parent in current.parents:
        if (parent / "20250115-p-adic-GRN-rewrite").exists():
            return parent
    raise RuntimeError("No se encontró la raíz del workspace.")


def _load_core_module() -> Any:
    root = _workspace_root()
    module_path = (
        root
        / "20250115-p-adic-GRN-rewrite"
        / "articulo-final"
        / "supplementary"
        / "code"
        / "core"
        / "compute_mu_generalized.py"
    )
    if not module_path.exists():
        raise FileNotFoundError(f"No existe el módulo core: {module_path}")
    spec =
    ↪  importlib.util.spec_from_file_location("core_compute_mu_generalized",
    ↪  module_path)
    if spec is None or spec.loader is None:
        raise RuntimeError("No se pudo cargar el módulo core de referencia.")
    module = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(module)
    return module


def _parse_ordering(ordering_text: str, n: int) -> list[int]:
    order = [int(x) for x in ordering_text.replace(",", " ").split()]
    if sorted(order) != list(range(n)):
        raise ValueError(f"ordering inválido; debe ser permutación de
        ↪  0..{n-1}")
    return order


def _write_json(output_path: Path, payload: dict[str, Any]) -> None:
    output_path.parent.mkdir(parents=True, exist_ok=True)
    with output_path.open("w", encoding="utf-8") as f:
        json.dump(payload, f, indent=2, ensure_ascii=False)


def main_score_ordering() -> None:
    parser = argparse.ArgumentParser(
        description="Calcula mu_E, mu_A, mu_I para un ordering dado."
    )
```

```
    parser.add_argument("transitions_csv", help="CSV con columnas
    ↪  config_from,config_to")
    parser.add_argument("N", type=int, help="Número de variables")
    parser.add_argument("p", type=int, help="Base discreta")
    parser.add_argument("ordering", help='Permutación, ej. "0 1 2 3" o
    ↪  "0,1,2,3"')
    parser.add_argument(
        "--output-json",
        default="scores.json",
        help="Archivo de salida JSON (default: scores.json)",
    )
    args = parser.parse_args()

    try:
        module = _load_core_module()
        table = module.read_transition_table_csv(args.transitions_csv, args.p)
        ordering = _parse_ordering(args.ordering, args.N)
        scores = module.compute_scores(args.p, args.N, ordering, table,
        ↪  return_profiles=True)
        payload = {
            "software": "padic-grn-core",
            "version": "0.1.0",
            "N": args.N,
            "p": args.p,
            "ordering": ordering,
            "scores": scores,
        }
        out = Path(args.output_json).resolve()
        _write_json(out, payload)
        print(f"OK: resultados guardados en {out}")
    except Exception as exc:  # pragma: no cover - CLI guard
        print(f"ERROR pgcore-score-ordering: {exc}", file=sys.stderr)
        raise SystemExit(1) from exc


def main_batch_scores() -> None:
    parser = argparse.ArgumentParser(
        description="Calcula scores para una lista de orderings (batch)."
    )
    parser.add_argument("transitions_csv", help="CSV con transiciones")
    parser.add_argument("N", type=int, help="Número de variables")
    parser.add_argument("p", type=int, help="Base discreta")
    parser.add_argument(
        "orderings_csv",
        help="CSV con columna ordering (string con enteros separados por
        ↪  espacio/coma)",
    )
    parser.add_argument(
        "--output-csv",
        default="batch_scores.csv",
        help="Archivo de salida CSV (default: batch_scores.csv)",
    )
    args = parser.parse_args()

    try:
```

```
        module = _load_core_module()
        table = module.read_transition_table_csv(args.transitions_csv, args.p)

        rows_out: list[dict[str, Any]] = []
        with open(args.orderings_csv, "r", encoding="utf-8") as f:
            reader = csv.DictReader(f)
            if "ordering" not in (reader.fieldnames or []):
                raise ValueError("orderings_csv debe contener columna
                ↪  'ordering'")
            for row in reader:
                ordering_raw = row["ordering"].strip()
                ordering = _parse_ordering(ordering_raw, args.N)
                scores = module.compute_scores(args.p, args.N, ordering,
                ↪  table, return_profiles=False)
                rows_out.append(
                    {
                        "ordering": " ".join(str(x) for x in ordering),
                        "mu_E": scores["mu_E"],
                        "mu_A": scores["mu_A"],
                        "mu_I": scores["mu_I"],
                    }
                )

        out_path = Path(args.output_csv).resolve()
        out_path.parent.mkdir(parents=True, exist_ok=True)
        with out_path.open("w", encoding="utf-8", newline="") as f:
            writer = csv.DictWriter(f, fieldnames=["ordering", "mu_E", "mu_A",
            ↪  "mu_I"])
            writer.writeheader()
            writer.writerows(rows_out)
        print(f"OK: batch guardado en {out_path} ({len(rows_out)} filas)")
    except Exception as exc:  # pragma: no cover - CLI guard
        print(f"ERROR pgcore-batch-scores: {exc}", file=sys.stderr)
        raise SystemExit(1) from exc

"""Adaptadores de entrada para padic-grn-core."""

from __future__ import annotations

from typing import Any


def from_grn_table(transitions: dict[int, int], n: int, p: int) -> dict[str,
↪  Any]:
    """Adaptador oficial para GRN discretas deterministas.

    Retorna una estructura normalizada para el núcleo de cálculo.
    """
    expected = p**n
    return {
        "kind": "grn_table",
        "N": n,
        "p": p,
        "expected_states": expected,
        "num_transitions": len(transitions),
```

```
        "transitions": transitions,
    }


def from_discrete_system(
    transitions: dict[int, int],
    n: int,
    p: int,
    metadata: dict[str, Any] | None = None,
) -> dict[str, Any]:
    """Adaptador para sistemas discretos generales.

    Esta interfaz asume codificación a espacio uniforme p^N.
    """
    payload = from_grn_table(transitions, n, p)
    payload["kind"] = "discrete_system"
    payload["metadata"] = metadata or {}
    return payload

"""Valida valores esperados contra evidencia JSON generada."""

from __future__ import annotations

import json
from dataclasses import dataclass
from pathlib import Path
from typing import Dict, List, Tuple


@dataclass
class CaseSpec:
    name: str
    json_path: Path
    expected_scores: Dict[str, int]
    n: int
    p: int


def load_json(path: Path) -> dict:
    with path.open("r", encoding="utf-8") as f:
        return json.load(f)


def check_case(case: CaseSpec) -> Tuple[bool, List[str]]:
    payload = load_json(case.json_path)
    scores = payload["scores"]
    issues: List[str] = []

    # Exact score checks
    for key, expected in case.expected_scores.items():
        got = scores.get(key)
        if got != expected:
            issues.append(f"{key}: esperado={expected}, obtenido={got}")

    # Global identity: mu_E + mu_A + mu_I = (N-1) * p^N
```

```
    left = scores["mu_E"] + scores["mu_A"] + scores["mu_I"]
    right = (case.n - 1) * (case.p**case.n)
    if left != right:
        issues.append(f"identidad_global: esperado={right}, obtenido={left}")

    # Per-level identity: E(n)+A(n)+I(n)=p^n
    E = scores.get("E", {})
    A = scores.get("A", {})
    I = scores.get("I", {})
    for level in range(1, case.n):
        k = str(level)
        got = int(E.get(k, 0)) + int(A.get(k, 0)) + int(I.get(k, 0))
        expected = case.p**level
        if got != expected:
            issues.append(f"identidad_nivel_{level}: esperado={expected},
             ↪  obtenido={got}")

    return (len(issues) == 0), issues


def make_report(cases: List[CaseSpec], out_path: Path) -> None:
    lines = [
        "# Validación cruzada automática vs valores esperados",
        "",
        "Fecha: 2026-04-25",
        "",
        "Este reporte compara los JSON de evidencia con valores esperados de
         ↪  artículo y checks matemáticos.",
        "",
    ]

    global_ok = True
    for case in cases:
        ok, issues = check_case(case)
        status = "PASS" if ok else "FAIL"
        lines.append(f"## Caso: {case.name} — {status}")
        lines.append("")
        lines.append(f"- Archivo: `{case.json_path}`")
        lines.append(f"- Esperados: `{case.expected_scores}`")
        if ok:
            lines.append("- Resultado: todos los checks pasaron.")
        else:
            global_ok = False
            lines.append("- Issues:")
            for issue in issues:
                lines.append(f"  - {issue}")
        lines.append("")

    lines.append(f"## Resultado global: {'PASS' if global_ok else 'FAIL'}")
    lines.append("")
    out_path.parent.mkdir(parents=True, exist_ok=True)
    out_path.write_text("\n".join(lines), encoding="utf-8")


def main() -> None:
```

```
    root = Path(__file__).resolve()
    workspace = None
    for p in root.parents:
        if (p / "registro-dossier").exists():
            workspace = p
            break
    if workspace is None:
        raise RuntimeError("No se encontró raíz del workspace.")

    ev = workspace / "registro-dossier" / "06_evidencia_corridas"
    cases = [
        CaseSpec(
            name="Athaliana pi* (N=13, p=2)",
            json_path=ev / "core_scores_athaliana_pi_star.json",
            expected_scores={"mu_E": 26776, "mu_A": 53880, "mu_I": 17648},
            n=13,
            p=2,
        ),
        CaseSpec(
            name="C. elegans pi* (N=8, p=2)",
            json_path=ev / "core_scores_celegans_pi_star.json",
            expected_scores={"mu_E": 956, "mu_A": 392, "mu_I": 444},
            n=8,
            p=2,
        ),
        CaseSpec(
            name="Drosophila pi* (N=6, p=2)",
            json_path=ev / "core_scores_drosophila_pi_star.json",
            expected_scores={"mu_E": 64, "mu_A": 156, "mu_I": 100},
            n=6,
            p=2,
        ),
        CaseSpec(
            name="Toy identidad p=2, N=2",
            json_path=ev / "core_scores_p2.json",
            expected_scores={"mu_E": 0, "mu_A": 0, "mu_I": 4},
            n=2,
            p=2,
        ),
        CaseSpec(
            name="Toy identidad p=3, N=2",
            json_path=ev / "core_scores_p3.json",
            expected_scores={"mu_E": 0, "mu_A": 0, "mu_I": 9},
            n=2,
            p=3,
        ),
    ]
    out = workspace / "registro-dossier" / "06_evidencia_corridas" /
    ↪  "VALIDACION-CRUZADA-VALORES.md"
    make_report(cases, out)
    print(f"OK: reporte generado en {out}")


if __name__ == "__main__":
    main()
```

8.9. Pruebas y ejemplos

```
from padic_grn_core import __version__


def test_version_defined() -> None:
    assert isinstance(__version__, str)
    assert __version__ == "0.1.0"

from padic_grn_core.adapters import from_discrete_system, from_grn_table


def test_from_grn_table_shape() -> None:
    payload = from_grn_table({0: 0, 1: 1, 2: 2, 3: 3}, n=2, p=2)
    assert payload["kind"] == "grn_table"
    assert payload["expected_states"] == 4
    assert payload["num_transitions"] == 4


def test_from_discrete_system_metadata() -> None:
    meta = {"source": "unit-test"}
    payload = from_discrete_system({0: 0, 1: 1, 2: 2, 3: 3}, n=2, p=2,
    ↪  metadata=meta)
    assert payload["kind"] == "discrete_system"
    assert payload["metadata"]["source"] == "unit-test"
```

```
config_from,config_to
0,0
1,1
2,2
3,3
```

```
config_from,config_to
0,0
1,1
2,2
3,3
4,4
5,5
6,6
7,7
8,8
```

```
ordering
0 1
1 0
```

8.10. Evidencia de validación

```
...
↪  [100%]
3 passed in 0.01s
```

```
{
  "software": "padic-grn-core",
  "version": "0.1.0",
```

```
  "N": 2,
  "p": 2,
  "ordering": [
    0,
    1
  ],
  "scores": {
    "mu_E": 0,
    "mu_A": 0,
    "mu_I": 4,
    "p": 2,
    "N": 2,
    "E": {
      "1": 0
    },
    "A": {
      "1": 0
    },
    "I": {
      "1": 2
    }
  }
}

{
  "software": "padic-grn-core",
  "version": "0.1.0",
  "N": 2,
  "p": 3,
  "ordering": [
    0,
    1
  ],
  "scores": {
    "mu_E": 0,
    "mu_A": 0,
    "mu_I": 9,
    "p": 3,
    "N": 2,
    "E": {
      "1": 0
    },
    "A": {
      "1": 0
    },
    "I": {
      "1": 3
    }
  }
}

ordering,mu_E,mu_A,mu_I
0 1,0,0,4
1 0,0,0,4
```

9. Sección II: Módulo search (búsqueda/certificación)

9.1. Programas de optimización incluidos

- `pgsearch-ga`: búsqueda heurística con objetivo seleccionable (minimizar `mu_E` o maximizar `mu_A`).
- `pgsearch-bb-min-muE`: branch-and-bound/backtracking exacto para minimizar `mu_E`, con poda por cota parcial cuando está disponible.
- `pgsearch-bb-max-muA`: búsqueda exacta por backtracking para maximizar `mu_A`.

9.2. README

```
# padic-grn-search: Búsqueda y certificación de orderings p-ádicos

## Resumen

Paquete de optimización y certificación de ordenamientos (`GA` + `B&B`) sobre
↪  el modelo discreto definido en `padic-grn-core`.

## Introducción

`padic-grn-search` amplía el uso del núcleo matemático para explorar múltiples
↪  orderings de manera sistemática. Permite comparar candidatos y certificar
↪  soluciones en escenarios donde la búsqueda exhaustiva o heurística es
↪  necesaria.

## Comandos

- `pgsearch-ga`
- `pgsearch-bb-min-muE`
- `pgsearch-bb-max-muA`

## Implementación de optimización y certificación

La implementación actual incluye:

- `pgsearch-ga`: algoritmo genético con selección por torneo, cruce PMX,
  mutación por intercambio y élite.
- `pgsearch-bb-min-muE`: búsqueda por branch-and-bound/backtracking exacto
↪  para
  minimizar `mu_E`, con poda por cota parcial cuando está disponible.
- `pgsearch-bb-max-muA`: búsqueda exacta por backtracking para maximizar
  `mu_A`.

El cálculo de métricas se realiza con el núcleo matemático de referencia para
mantener continuidad y trazabilidad científica.

## Autoría oficial

1. Jesús Rogelio Pérez-Buendía (SECIHTI-CIMAT, Unidad Mérida) — ORCID
↪  `0000-0002-7739-4779`
2. Víctor Nopal-Coello (Universidad Autónoma del Estado de Hidalgo, UAEH) —
↪  ORCID `0000-0003-2608-3636`
```

````
3. Yuriria Cortés-Poza (IIMAS, UNAM – Unidad Académica Yucatán) – ORCID
↪  `0000-0003-1856-9206`

## DOI

- DOI del software: `DOI_SOFTWARE_SEARCH`.

## Financiamiento

Modelos matemáticos y computacionales no convencionales para el estudio y
↪  análisis de problemas relevantes en Biología, CF 2019/217367.
````

9.3. Manual de usuario

````
# Manual de usuario – padic-grn-search

## Instalación

```bash
cd software-suite/padic-grn-search
pip install -e .
```

## Comandos principales

### 1) GA

```bash
pgsearch-ga ../padic-grn-core/examples/transitions_p2_n2_identity.csv 2 2
↪  --objective minimize_mu_E --generations 5 --population 10 --seed 7
```

### 2) B&B mínimo de mu_E

```bash
pgsearch-bb-min-muE ../padic-grn-core/examples/transitions_p2_n2_identity.csv
↪  2 2
```

### 3) B&B máximo de mu_A

```bash
pgsearch-bb-max-muA ../padic-grn-core/examples/transitions_p2_n2_identity.csv
↪  2 2
```

## Recomendaciones

- Para pruebas rápidas usar redes pequeñas (`N<=8`).
- Para certificación exacta usar `pgsearch-bb-*`.
- Para exploración rápida en `N` grande usar `pgsearch-ga` y fijar `--seed`.
- Registrar siempre parámetros y versión del software.
````

9.4. Manual técnico

```
# Manual técnico — padic-grn-search

## 1. Propósito

`padic-grn-search` resuelve optimización y certificación de ordenamientos:

- optimización heurística (GA),
- certificación exacta/semiexacta (B&B, según configuración).

## 2. Arquitectura actual

- `src/padic_grn_search/engine.py`
  - `random_ga_search(...)` (GA con torneo + PMX + mutación + élite),
  - `branch_and_bound_search(...)` (certificación exacta por árbol de
  ↪  búsqueda),
  - `exhaustive_search(...)` (búsqueda completa, útil para verificación).
- `src/padic_grn_search/cli.py`
  - `pgsearch-ga`
  - `pgsearch-bb-min-muE`
  - `pgsearch-bb-max-muA`

La capa de búsqueda es nativa; el cálculo de métricas usa el núcleo matemático
de referencia para conservar continuidad científica.

## 3. Diseño algorítmico (resumen)

- GA nativo:
  - selección por torneo,
  - cruce PMX para permutaciones válidas,
  - mutación por intercambio,
  - élite por generación,
  - evaluación por objetivo (`mu_E` o `-mu_A`) con cache de evaluaciones.
- Branch-and-Bound / backtracking exacto:
  - para `minimize_mu_E` usa poda por cota parcial cuando está disponible,
  - para `maximize_mu_A` ejecuta búsqueda exacta por backtracking.
- Exhaustivo nativo:
  - recorre `N!` permutaciones,
  - opción de truncamiento por presupuesto de evaluaciones.

## 4. Entrada/Salida

Entrada:

- transiciones (`csv`),
- `N`, `p`,
- parámetros de corrida (`generations`, `population`, `seed`,
↪  `max-evaluations`).

Salida:

- mejor ordenamiento,
- métricas (`mu_E`, `mu_A`, `mu_I`),
- conteo de evaluaciones,
```

```
- conteo de ramas podadas (en B&B),
- indicador de truncamiento (si aplica).

## 5. Trazabilidad de ejecución

- fijar `--seed` en corridas de GA para reproducibilidad,
- registrar parámetros de entrada y objetivo,
- conservar JSON de salida de cada corrida como evidencia verificable.
```

9.5. Configuración de paquete

```
[build-system]
requires = ["setuptools>=68", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "padic-grn-search"
version = "0.1.0"
description = "Busqueda y certificacion de ordenamientos para sistemas
↪  discretos."
readme = "README.md"
requires-python = ">=3.10"
license = {text = "Licencia propietaria para uso academico no comercial"}
authors = [
  {name = "Jesús Rogelio Pérez-Buendía", email = "rogelio.perez@cimat.mx"},
  {name = "Víctor Nopal-Coello", email = "victor_nopal@uaeh.edu.mx"},
  {name = "Yuriria Cortés-Poza", email = "yuriria.cortes@iimas.unam.mx"}
]
dependencies = []

[project.scripts]
pgsearch-ga = "padic_grn_search.cli:main_ga"
pgsearch-bb-min-muE = "padic_grn_search.cli:main_bb_min_mue"
pgsearch-bb-max-muA = "padic_grn_search.cli:main_bb_max_mua"

[tool.setuptools]
package-dir = {"" = "src"}

[tool.setuptools.packages.find]
where = ["src"]
```

9.6. Cita académica

```
cff-version: 1.2.0
message: "If you use this software, please cite it as below."
title: "padic-grn-search"
version: "0.1.0"
type: software
authors:
  - family-names: "Pérez-Buendía"
    given-names: "Jesús Rogelio"
    affiliation: "SECIHTI - Centro de Investigación en Matemáticas (CIMAT),
    ↪  Unidad Mérida"
    orcid: "https://orcid.org/0000-0002-7739-4779"
  - family-names: "Nopal-Coello"
```

```
    given-names: "Víctor"
    affiliation: "Universidad Autónoma del Estado de Hidalgo (UAEH)"
    orcid: "https://orcid.org/0000-0003-2608-3636"
  - family-names: "Cortés-Poza"
    given-names: "Yuriria"
    affiliation: "Instituto de Investigaciones en Matemáticas Aplicadas y en
    ↪  Sistemas (IIMAS), UNAM -- Unidad Académica Yucatán"
    orcid: "https://orcid.org/0000-0003-1856-9206"
abstract: "Módulo de búsqueda y certificación de ordenamientos óptimos
↪  mediante algoritmos genéticos y branch-and-bound para sistemas dinámicos
↪  discretos."
license: "Licencia propietaria para uso academico no comercial"
repository-code: "PENDING_REPOSITORY_URL"
doi: "10.0000/pending-doi"
```

9.7. Licencia

```
Licencia propietaria para uso académico no comercial
Copyright (c) 2026
Jesús Rogelio Pérez-Buendía, Víctor Nopal-Coello, Yuriria Cortés-Poza

Permisos:
- Se autoriza el uso académico y de investigación sin fines de lucro.
- Se autoriza la evaluación técnica y científica del software.

Restricciones:
- No se autoriza la redistribución de copias sin permiso escrito.
- No se autoriza sublicenciar el software sin permiso escrito.
- No se autoriza el uso comercial sin permiso escrito.
- No se autoriza modificar y publicar versiones derivadas sin permiso escrito.

Contacto para autorizaciones:
- Jesús Rogelio Pérez-Buendía: rogelio.perez@cimat.mx

Esta licencia aplica a la versión actual del software y a los materiales
incluidos en este repositorio, salvo que un archivo específico indique
explícitamente otra condición.
```

9.8. Configuración de pruebas

```
[pytest]
pythonpath = src
```

9.9. Código fuente

```
"""padic-grn-search package."""

__all__ = ["__version__"]
__version__ = "0.1.0"

"""CLI funcional nativa para padic-grn-search."""

from __future__ import annotations

import argparse
```

```
import json
import sys
from pathlib import Path

from .engine import branch_and_bound_search, random_ga_search


def _write_json(path: str, payload: dict) -> Path:
    out = Path(path).resolve()
    out.parent.mkdir(parents=True, exist_ok=True)
    with out.open("w", encoding="utf-8") as f:
        json.dump(payload, f, indent=2, ensure_ascii=False)
    return out


def main_ga() -> None:
    parser = argparse.ArgumentParser(description="Búsqueda GA (torneo + PMX +
    ↪  mutación + élite).")
    parser.add_argument("transitions_csv")
    parser.add_argument("N", type=int)
    parser.add_argument("p", type=int)
    parser.add_argument("--objective", choices=["minimize_mu_E",
    ↪  "maximize_mu_A"], default="minimize_mu_E")
    parser.add_argument("--generations", type=int, default=50)
    parser.add_argument("--population", type=int, default=30)
    parser.add_argument("--seed", type=int, default=None)
    parser.add_argument("--output-json", default="search_ga_result.json")
    args = parser.parse_args()
    try:
        result = random_ga_search(
            transitions_csv=args.transitions_csv,
            n=args.N,
            p=args.p,
            objective=args.objective,
            generations=args.generations,
            population=args.population,
            seed=args.seed,
        )
        out = _write_json(args.output_json, result)
        print(f"OK: resultado GA guardado en {out}")
    except Exception as exc:  # pragma: no cover - CLI guard
        print(f"ERROR pgsearch-ga: {exc}", file=sys.stderr)
        raise SystemExit(1) from exc


def main_bb_min_mue() -> None:
    parser = argparse.ArgumentParser(description="Búsqueda Branch-and-Bound
    ↪  para minimizar mu_E.")
    parser.add_argument("transitions_csv")
    parser.add_argument("N", type=int)
    parser.add_argument("p", type=int)
    parser.add_argument("--max-evaluations", type=int, default=None)
    parser.add_argument("--output-json", default="search_bb_min_result.json")
    args = parser.parse_args()
    try:
```

```
        result = branch_and_bound_search(
            transitions_csv=args.transitions_csv,
            n=args.N,
            p=args.p,
            objective="minimize_mu_E",
            max_evaluations=args.max_evaluations,
        )
        out = _write_json(args.output_json, result)
        print(f"OK: resultado BB-min guardado en {out}")
    except Exception as exc:  # pragma: no cover - CLI guard
        print(f"ERROR pgsearch-bb-min-muE: {exc}", file=sys.stderr)
        raise SystemExit(1) from exc


def main_bb_max_mua() -> None:
    parser = argparse.ArgumentParser(description="Búsqueda Branch-and-Bound
    ↪  para maximizar mu_A.")
    parser.add_argument("transitions_csv")
    parser.add_argument("N", type=int)
    parser.add_argument("p", type=int)
    parser.add_argument("--max-evaluations", type=int, default=None)
    parser.add_argument("--output-json", default="search_bb_max_result.json")
    args = parser.parse_args()
    try:
        result = branch_and_bound_search(
            transitions_csv=args.transitions_csv,
            n=args.N,
            p=args.p,
            objective="maximize_mu_A",
            max_evaluations=args.max_evaluations,
        )
        out = _write_json(args.output_json, result)
        print(f"OK: resultado BB-max guardado en {out}")
    except Exception as exc:  # pragma: no cover - CLI guard
        print(f"ERROR pgsearch-bb-max-muA: {exc}", file=sys.stderr)
        raise SystemExit(1) from exc

"""Motor de búsqueda para padic-grn-search.

Incluye:
- Algoritmo genético con selección por torneo, PMX, mutación y élite.
- Branch-and-Bound / backtracking exacto para minimizar mu_E o maximizar mu_A.
- Búsqueda exhaustiva (baseline y verificación).
"""

from __future__ import annotations

import importlib.util
import itertools
import random
from pathlib import Path
from typing import Any


def workspace_root() -> Path:
```

```
    current = Path(__file__).resolve()
    for parent in current.parents:
        if (parent / "20250115-p-adic-GRN-rewrite").exists():
            return parent
    raise RuntimeError("No se encontró la raíz del workspace.")


def _core_candidate_paths(root: Path) -> list[Path]:
    return [
        root / "Paper-II.v3.2" / "supplementary" / "code" / "core" /
        ↪  "compute_mu_generalized.py",
        root
        / "20250115-p-adic-GRN-rewrite"
        / "articulo-final"
        / "supplementary"
        / "code"
        / "core"
        / "compute_mu_generalized.py",
    ]


def load_reference_core_module() -> Any:
    root = workspace_root()
    module_path = None
    for candidate in _core_candidate_paths(root):
        if candidate.exists():
            module_path = candidate
            break
    if module_path is None:
        raise FileNotFoundError("No se encontró compute_mu_generalized.py en
        ↪  rutas de referencia.")
    spec = importlib.util.spec_from_file_location("reference_core_module",
    ↪  module_path)
    if spec is None or spec.loader is None:
        raise RuntimeError("No se pudo crear spec del módulo core de
        ↪  referencia.")
    mod = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(mod)
    return mod


class _ReferenceEvaluator:
    def __init__(self, transitions_csv: str, n: int, p: int):
        self.core = load_reference_core_module()
        self.transitions_csv = transitions_csv
        self.n = n
        self.p = p
        self.table = self.core.read_transition_table_csv(transitions_csv, p)
        self.calculator = None
        if hasattr(self.core, "MuCalculator"):
            self.calculator = self.core.MuCalculator(N=n, p=p,
            ↪  transition_table=self.table)

    def score(self, ordering: list[int]) -> dict[str, int]:
        if self.calculator is not None:
```

```
            scores = self.calculator.compute_scores_cached(ordering,
            ↪  return_profiles=False)
        else:
            scores = self.core.compute_scores(self.p, self.n, ordering,
            ↪  self.table, return_profiles=False)
        return {"mu_E": scores["mu_E"], "mu_A": scores["mu_A"], "mu_I":
        ↪  scores["mu_I"]}

    def partial_mu_e(self, prefix: list[int]) -> int | None:
        if hasattr(self.core, "partial_mu_E"):
            return int(self.core.partial_mu_E(prefix, self.n, self.p,
            ↪  self.table))
        return None


def evaluate_ordering(
    transitions_csv: str,
    n: int,
    p: int,
    ordering: list[int],
) -> dict[str, int]:
    evaluator = _ReferenceEvaluator(transitions_csv, n, p)
    return evaluator.score(ordering)


def _fitness(scores: dict[str, int], objective: str) -> int:
    return scores["mu_E"] if objective == "minimize_mu_E" else -scores["mu_A"]


def _pmx_crossover(parent1: list[int], parent2: list[int], rng: random.Random)
↪  -> tuple[list[int], list[int]]:
    n = len(parent1)
    c1, c2 = sorted(rng.sample(range(n), 2))

    def build_child(p1: list[int], p2: list[int]) -> list[int]:
        child = [-1] * n
        child[c1 : c2 + 1] = p1[c1 : c2 + 1]
        for i in range(c1, c2 + 1):
            gene = p2[i]
            if gene in child:
                continue
            pos = i
            while c1 <= pos <= c2:
                pos = p2.index(p1[pos])
            child[pos] = gene
        for i in range(n):
            if child[i] == -1:
                child[i] = p2[i]
        return child

    return build_child(parent1, parent2), build_child(parent2, parent1)


def _mutate_swap(ordering: list[int], rng: random.Random, mutation_rate:
↪  float) -> list[int]:
```

```
    out = list(ordering)
    for i in range(len(out)):
        if rng.random() < mutation_rate:
            j = rng.randrange(len(out))
            out[i], out[j] = out[j], out[i]
    return out


def _tournament_select(
    population: list[list[int]],
    fitnesses: list[int],
    rng: random.Random,
    k: int = 3,
) -> list[int]:
    idxs = [rng.randrange(len(population)) for _ in range(k)]
    best_idx = min(idxs, key=lambda idx: fitnesses[idx])
    return list(population[best_idx])


def random_ga_search(
    transitions_csv: str,
    n: int,
    p: int,
    objective: str = "minimize_mu_E",
    generations: int = 50,
    population: int = 30,
    seed: int | None = None,
) -> dict[str, Any]:
    rng = random.Random(seed)
    evaluator = _ReferenceEvaluator(transitions_csv, n, p)

    mutation_rate = 0.1
    elite_size = max(1, min(5, population // 4))
    index_pool = list(range(n))

    pop = [rng.sample(index_pool, n) for _ in range(population)]
    score_cache: dict[tuple[int, ...], dict[str, int]] = {}

    def eval_order(ordering: list[int]) -> tuple[dict[str, int], int]:
        key = tuple(ordering)
        if key not in score_cache:
            score_cache[key] = evaluator.score(ordering)
        scores = score_cache[key]
        return scores, _fitness(scores, objective)

    evaluated = [eval_order(ind) for ind in pop]
    evals = len(score_cache)

    best_order = None
    best_scores = None
    best_fit = None
    history: list[int] = []

    for _ in range(generations):
        fitnesses = [fit for _, fit in evaluated]
```

```
        ranked_idxs = sorted(range(len(pop)), key=lambda i: fitnesses[i])
        pop = [pop[i] for i in ranked_idxs]
        evaluated = [evaluated[i] for i in ranked_idxs]
        fitnesses = [fit for _, fit in evaluated]

        if best_fit is None or fitnesses[0] < best_fit:
            best_fit = fitnesses[0]
            best_order = list(pop[0])
            best_scores = dict(evaluated[0][0])
        history.append(fitnesses[0])

        next_pop = [list(ind) for ind in pop[:elite_size]]
        while len(next_pop) < population:
            p1 = _tournament_select(pop, fitnesses, rng, k=3)
            p2 = _tournament_select(pop, fitnesses, rng, k=3)
            c1, c2 = _pmx_crossover(p1, p2, rng)
            c1 = _mutate_swap(c1, rng, mutation_rate)
            c2 = _mutate_swap(c2, rng, mutation_rate)
            next_pop.append(c1)
            if len(next_pop) < population:
                next_pop.append(c2)

        pop = next_pop
        evaluated = [eval_order(ind) for ind in pop]
        evals = len(score_cache)

    return {
        "method": "ga_pmx_tournament_elite",
        "objective": objective,
        "evaluations": evals,
        "best_ordering": best_order,
        "best_fitness": best_fit,
        "best_scores": best_scores,
        "history": history,
        "seed": seed,
    }


def exhaustive_search(
    transitions_csv: str,
    n: int,
    p: int,
    objective: str = "minimize_mu_E",
    max_evaluations: int | None = None,
) -> dict[str, Any]:
    evaluator = _ReferenceEvaluator(transitions_csv, n, p)

    best_order = None
    best_fit = None
    best_scores = None
    evals = 0

    for perm in itertools.permutations(range(n)):
        if max_evaluations is not None and evals >= max_evaluations:
            break
```

```
        scores = evaluator.score(list(perm))
        fit = _fitness(scores, objective)
        evals += 1
        if best_fit is None or fit < best_fit:
            best_fit = fit
            best_order = list(perm)
            best_scores = scores

    return {
        "method": "native_exhaustive",
        "objective": objective,
        "evaluations": evals,
        "best_ordering": best_order,
        "best_fitness": best_fit,
        "best_scores": best_scores,
        "truncated": max_evaluations is not None and evals >= max_evaluations,
    }


def branch_and_bound_search(
    transitions_csv: str,
    n: int,
    p: int,
    objective: str = "minimize_mu_E",
    max_evaluations: int | None = None,
) -> dict[str, Any]:
    evaluator = _ReferenceEvaluator(transitions_csv, n, p)
    used = [False] * n
    current = [0] * n
    evals = 0
    pruned = 0

    best_fit = float("inf")
    best_order: list[int] | None = None
    best_scores: dict[str, int] | None = None

    use_partial_bound = objective == "minimize_mu_E" and
    ↪  evaluator.partial_mu_e([0]) is not None

    def backtrack(depth: int) -> None:
        nonlocal evals, pruned, best_fit, best_order, best_scores

        if max_evaluations is not None and evals >= max_evaluations:
            return

        if depth == n:
            scores = evaluator.score(current)
            fit = _fitness(scores, objective)
            evals += 1
            if fit < best_fit:
                best_fit = fit
                best_order = list(current)
                best_scores = scores
            return
```

```
        for g in range(n):
            if used[g]:
                continue
            current[depth] = g
            if use_partial_bound:
                mu_k = evaluator.partial_mu_e(current[: depth + 1])
                if mu_k is not None and mu_k > best_fit:
                    pruned += 1
                    continue
            used[g] = True
            backtrack(depth + 1)
            used[g] = False

    backtrack(0)
    return {
        "method": "branch_and_bound_partial_muE" if use_partial_bound else
        ↪  "backtracking_exact",
        "objective": objective,
        "evaluations": evals,
        "pruned": pruned,
        "best_ordering": best_order,
        "best_fitness": best_fit if best_fit != float("inf") else None,
        "best_scores": best_scores,
        "truncated": max_evaluations is not None and evals >= max_evaluations,
    }
```

9.10. Pruebas

```
from padic_grn_search import __version__


def test_version_defined() -> None:
    assert isinstance(__version__, str)
    assert __version__ == "0.1.0"

from __future__ import annotations

from pathlib import Path

from padic_grn_search.engine import branch_and_bound_search,
↪  exhaustive_search, random_ga_search


def _write_identity_csv(path: Path, p: int, n: int) -> None:
    total = p**n
    rows = ["config_from,config_to"]
    rows.extend(f"{i},{i}" for i in range(total))
    path.write_text("\n".join(rows), encoding="utf-8")


def test_random_ga_search_returns_result(tmp_path: Path) -> None:
    csv_path = tmp_path / "transitions.csv"
    _write_identity_csv(csv_path, p=2, n=2)
    out = random_ga_search(
        transitions_csv=str(csv_path),
```

```
        n=2,
        p=2,
        generations=2,
        population=4,
        seed=7,
    )
    assert out["best_ordering"] in ([0, 1], [1, 0])
    assert set(out["best_scores"].keys()) == {"mu_E", "mu_A", "mu_I"}


def test_exhaustive_search_truncated(tmp_path: Path) -> None:
    csv_path = tmp_path / "transitions.csv"
    _write_identity_csv(csv_path, p=2, n=3)
    out = exhaustive_search(
        transitions_csv=str(csv_path),
        n=3,
        p=2,
        max_evaluations=2,
    )
    assert out["evaluations"] == 2
    assert out["truncated"] is True


def test_branch_and_bound_search_returns_result(tmp_path: Path) -> None:
    csv_path = tmp_path / "transitions.csv"
    _write_identity_csv(csv_path, p=2, n=2)
    out = branch_and_bound_search(
        transitions_csv=str(csv_path),
        n=2,
        p=2,
        objective="minimize_mu_E",
    )
    assert out["best_ordering"] in ([0, 1], [1, 0])
    assert out["best_scores"]["mu_E"] == 0
```

9.11. Evidencia de validación

```
...
↪  [100%]
3 passed in 0.02s

{
  "method": "ga_pmx_tournament_elite",
  "objective": "minimize_mu_E",
  "evaluations": 2,
  "best_ordering": [
    1,
    0
  ],
  "best_fitness": 0,
  "best_scores": {
    "mu_E": 0,
    "mu_A": 0,
    "mu_I": 4
  },
```

```
  "history": [
    0,
    0,
    0,
    0,
    0
  ],
  "seed": 7
}

{
  "method": "branch_and_bound_partial_muE",
  "objective": "minimize_mu_E",
  "evaluations": 2,
  "pruned": 0,
  "best_ordering": [
    0,
    1
  ],
  "best_fitness": 0,
  "best_scores": {
    "mu_E": 0,
    "mu_A": 0,
    "mu_I": 4
  },
  "truncated": false
}

{
  "method": "backtracking_exact",
  "objective": "maximize_mu_A",
  "evaluations": 2,
  "pruned": 0,
  "best_ordering": [
    0,
    1
  ],
  "best_fitness": 0,
  "best_scores": {
    "mu_E": 0,
    "mu_A": 0,
    "mu_I": 4
  },
  "truncated": false
}
```


10. Sección III: Módulo viz (visualización/reportes)

10.1. README

```
# padic-grn-viz: Visualización y reportes de análisis p-ádico

## Resumen
```

```
Paquete de visualización y reportes para resultados producidos por
↪  `padic-grn-core` y `padic-grn-search`.

## Introducción

`padic-grn-viz` facilita la lectura de resultados con salidas gráficas
↪  reproducibles. Su objetivo es hacer más amigable la interpretación de
↪  métricas p-ádicas para usuarios técnicos y no técnicos.

## Comandos

- `pgviz-dependency-graph`
- `pgviz-hierarchical-graph`

## Alcance operativo

La versión actual implementa motores nativos para:

- gráfica de barras de `mu_E`, `mu_A`, `mu_I`,
- gráfica de posiciones del ordering.

Incluye plantillas visuales para reporte institucional.

## Autoría oficial

1. Jesús Rogelio Pérez-Buendía (SECIHTI-CIMAT, Unidad Mérida) — ORCID
↪  `0000-0002-7739-4779`
2. Víctor Nopal-Coello (Universidad Autónoma del Estado de Hidalgo, UAEH) —
↪  ORCID `0000-0003-2608-3636`
3. Yuriria Cortés-Poza (IIMAS, UNAM — Unidad Académica Yucatán) — ORCID
↪  `0000-0003-1856-9206`

## DOI

- DOI del software: `DOI_SOFTWARE_VIZ`.

## Financiamiento

Modelos matemáticos y computacionales no convencionales para el estudio y
↪  análisis de problemas relevantes en Biología, CF 2019/217367.
```

10.2. Manual de usuario

```
# Manual de usuario — padic-grn-viz

## Instalación

```bash
cd software-suite/padic-grn-viz
pip install -e .
```

Comandos

Dependencias
```

```bash
pgviz-dependency-graph
```

Jerárquico

```bash
pgviz-hierarchical-graph
```

Recomendaciones

- ejecutar desde entorno con `matplotlib` y `networkx` instalados.
- guardar resultados visuales junto con metadatos de corrida.

10.3. Manual técnico

```
# Manual técnico — padic-grn-viz

## 1. Propósito

`padic-grn-viz` transforma resultados numéricos (`core/search`) en artefactos
↪  visuales reproducibles para:

- análisis,
- suplementario de artículos,
- demostración web institucional.

## 2. Arquitectura actual

- `src/padic_grn_viz/engine.py`
  - carga de resultados (`load_scores_json`),
  - gráfico de barras de métricas (`plot_score_bars`),
  - gráfico de ordering (`plot_ordering_positions`).
- `src/padic_grn_viz/cli.py`
  - `pgviz-dependency-graph`
  - `pgviz-hierarchical-graph`

## 3. Tipos de salida

- figuras `png/pdf`,
- resúmenes tabulares de apoyo.

## 4. Contrato mínimo recomendado para integración

- entradas en `csv/json` con:
  - metadata (`N`, `p`, red, fecha),
  - métricas (`mu_E`, `mu_A`, `mu_I`),
  - ordering usado.

## 5. Roadmap técnico

1. Añadir gráficos avanzados de perfiles por nivel (`E/A/I`).
2. Añadir plantillas de reporte visual unificado (PNG + PDF + resumen).
```

3. Integración directa con backend de demo institucional.

10.4. Configuración de paquete

```
[build-system]
requires = ["setuptools>=68", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "padic-grn-viz"
version = "0.1.0"
description = "Visualizacion y reportes reproducibles para analisis p-adicos
↪  discretos."
readme = "README.md"
requires-python = ">=3.10"
license = {text = "Licencia propietaria para uso academico no comercial"}
authors = [
  {name = "Jesús Rogelio Pérez-Buendía", email = "rogelio.perez@cimat.mx"},
  {name = "Víctor Nopal-Coello", email = "victor_nopal@uaeh.edu.mx"},
  {name = "Yuriria Cortés-Poza", email = "yuriria.cortes@iimas.unam.mx"}
]
dependencies = ["matplotlib>=3.8"]

[project.scripts]
pgviz-dependency-graph = "padic_grn_viz.cli:main_dependency_graph"
pgviz-hierarchical-graph = "padic_grn_viz.cli:main_hierarchical_graph"

[tool.setuptools]
package-dir = {"" = "src"}

[tool.setuptools.packages.find]
where = ["src"]
```

10.5. Cita académica

```
cff-version: 1.2.0
message: "If you use this software, please cite it as below."
title: "padic-grn-viz"
version: "0.1.0"
type: software
authors:
  - family-names: "Pérez-Buendía"
    given-names: "Jesús Rogelio"
    affiliation: "SECIHTI - Centro de Investigación en Matemáticas (CIMAT),
    ↪  Unidad Mérida"
    orcid: "https://orcid.org/0000-0002-7739-4779"
  - family-names: "Nopal-Coello"
    given-names: "Víctor"
    affiliation: "Universidad Autónoma del Estado de Hidalgo (UAEH)"
    orcid: "https://orcid.org/0000-0003-2608-3636"
  - family-names: "Cortés-Poza"
    given-names: "Yuriria"
    affiliation: "Instituto de Investigaciones en Matemáticas Aplicadas y en
    ↪  Sistemas (IIMAS), UNAM -- Unidad Académica Yucatán"
    orcid: "https://orcid.org/0000-0003-1856-9206"
```

```
abstract: "Módulo de visualización y reportes reproducibles para resultados de
↪  análisis p-ádico en sistemas dinámicos discretos."
license: "Licencia propietaria para uso academico no comercial"
repository-code: "PENDING_REPOSITORY_URL"
doi: "10.0000/pending-doi"
```

10.6. Licencia

```
Licencia propietaria para uso académico no comercial
Copyright (c) 2026
Jesús Rogelio Pérez-Buendía, Víctor Nopal-Coello, Yuriria Cortés-Poza

Permisos:
- Se autoriza el uso académico y de investigación sin fines de lucro.
- Se autoriza la evaluación técnica y científica del software.

Restricciones:
- No se autoriza la redistribución de copias sin permiso escrito.
- No se autoriza sublicenciar el software sin permiso escrito.
- No se autoriza el uso comercial sin permiso escrito.
- No se autoriza modificar y publicar versiones derivadas sin permiso escrito.

Contacto para autorizaciones:
- Jesús Rogelio Pérez-Buendía: rogelio.perez@cimat.mx

Esta licencia aplica a la versión actual del software y a los materiales
incluidos en este repositorio, salvo que un archivo específico indique
explícitamente otra condición.
```

10.7. Configuración de pruebas

```
[pytest]
pythonpath = src
```

10.8. Código fuente

```
"""padic-grn-viz package."""

__all__ = ["__version__"]
__version__ = "0.1.0"
```

```
"""CLI funcional nativa para padic-grn-viz."""

from __future__ import annotations

import argparse
import sys

from .engine import load_scores_json, plot_ordering_positions, plot_score_bars


def main_dependency_graph() -> None:
    parser = argparse.ArgumentParser(
        description="Genera figura de barras para mu_E, mu_A, mu_I."
    )
```

```
    parser.add_argument("scores_json", help="Archivo JSON de resultados
    ↪  (core/search)")
    parser.add_argument("--output", default="viz_dependency_scores.png",
    ↪  help="Salida PNG")
    args = parser.parse_args()
    try:
        payload = load_scores_json(args.scores_json)
        out = plot_score_bars(payload, args.output)
        print(f"OK: figura guardada en {out}")
    except Exception as exc:  # pragma: no cover - CLI guard
        print(f"ERROR pgviz-dependency-graph: {exc}", file=sys.stderr)
        raise SystemExit(1) from exc


def main_hierarchical_graph() -> None:
    parser = argparse.ArgumentParser(
        description="Genera figura de posiciones del ordering."
    )
    parser.add_argument("scores_json", help="Archivo JSON de resultados
    ↪  (core/search)")
    parser.add_argument("--output", default="viz_hierarchical_ordering.png",
    ↪  help="Salida PNG")
    args = parser.parse_args()
    try:
        payload = load_scores_json(args.scores_json)
        out = plot_ordering_positions(payload, args.output)
        print(f"OK: figura guardada en {out}")
    except Exception as exc:  # pragma: no cover - CLI guard
        print(f"ERROR pgviz-hierarchical-graph: {exc}", file=sys.stderr)
        raise SystemExit(1) from exc

"""Motor nativo inicial para visualización en padic-grn-viz."""

from __future__ import annotations

import json
from pathlib import Path
from typing import Any

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt


def load_scores_json(path: str) -> dict[str, Any]:
    src = Path(path).resolve()
    with src.open("r", encoding="utf-8") as f:
        payload = json.load(f)
    return payload


def plot_score_bars(payload: dict[str, Any], output_path: str) -> Path:
    scores = payload.get("scores", payload.get("best_scores", {}))
    mu_e = scores.get("mu_E", 0)
```

```
    mu_a = scores.get("mu_A", 0)
    mu_i = scores.get("mu_I", 0)

    fig, ax = plt.subplots(figsize=(6, 4))
    ax.bar(["mu_E", "mu_A", "mu_I"], [mu_e, mu_a, mu_i], color=["#2E86AB",
    ↪  "#6A994E", "#7D53DE"])
    ax.set_title("Resumen de métricas p-ádicas")
    ax.set_ylabel("Valor")
    fig.tight_layout()

    out = Path(output_path).resolve()
    out.parent.mkdir(parents=True, exist_ok=True)
    fig.savefig(out, dpi=160)
    plt.close(fig)
    return out


def plot_ordering_positions(payload: dict[str, Any], output_path: str) ->
↪  Path:
    ordering = payload.get("ordering", payload.get("best_ordering", []))
    if not ordering:
        ordering = []

    fig, ax = plt.subplots(figsize=(7, 3))
    if ordering:
        ax.plot(range(len(ordering)), ordering, marker="o")
    ax.set_title("Posición vs índice de gen (ordering)")
    ax.set_xlabel("Posición")
    ax.set_ylabel("Índice de gen")
    fig.tight_layout()

    out = Path(output_path).resolve()
    out.parent.mkdir(parents=True, exist_ok=True)
    fig.savefig(out, dpi=160)
    plt.close(fig)
    return out
```

10.9. Pruebas

```
from padic_grn_viz import __version__


def test_version_defined() -> None:
    assert isinstance(__version__, str)
    assert __version__ == "0.1.0"

from __future__ import annotations

import json
from pathlib import Path

from padic_grn_viz.engine import load_scores_json, plot_ordering_positions,
↪  plot_score_bars
```

```
def test_load_scores_json(tmp_path: Path) -> None:
    src = tmp_path / "scores.json"
    src.write_text(json.dumps({"scores": {"mu_E": 1, "mu_A": 2, "mu_I": 3}}),
    ↪  encoding="utf-8")
    payload = load_scores_json(str(src))
    assert payload["scores"]["mu_E"] == 1


def test_plot_outputs(tmp_path: Path) -> None:
    payload = {
        "scores": {"mu_E": 1, "mu_A": 2, "mu_I": 3},
        "ordering": [0, 2, 1],
    }
    dep = plot_score_bars(payload, str(tmp_path / "dep.png"))
    hie = plot_ordering_positions(payload, str(tmp_path / "hie.png"))
    assert dep.exists()
    assert hie.exists()
```

10.10. Evidencia de validación

```
...
↪  [100%]
============================== warnings summary
↪  ==============================
../../../../Library/Python/3.9/lib/python/site-packages/matplotlib/_fontconfi⌋
↪  g_pattern.py:64
  /Users/yoyontzin/Library/Python/3.9/lib/python/site-packages/matplotlib/_fo⌋
   ↪  ntconfig_pattern.py:64: PyparsingDeprecationWarning: 'oneOf' deprecated
   ↪  - use 'one_of'
    prop = Group((name + Suppress("=") + comma_separated(value)) |
     ↪  oneOf(_CONSTANTS))

../../../../Library/Python/3.9/lib/python/site-packages/matplotlib/_fontconfi⌋
↪  g_pattern.py:85
../../../../Library/Python/3.9/lib/python/site-packages/matplotlib/_fontconfi⌋
↪  g_pattern.py:85
../../../../Library/Python/3.9/lib/python/site-packages/matplotlib/_fontconfi⌋
↪  g_pattern.py:85
../../../../Library/Python/3.9/lib/python/site-packages/matplotlib/_fontconfi⌋
↪  g_pattern.py:85
../../../../Library/Python/3.9/lib/python/site-packages/matplotlib/_fontconfi⌋
↪  g_pattern.py:85
../../../../Library/Python/3.9/lib/python/site-packages/matplotlib/_fontconfi⌋
↪  g_pattern.py:85
  /Users/yoyontzin/Library/Python/3.9/lib/python/site-packages/matplotlib/_fo⌋
   ↪  ntconfig_pattern.py:85: PyparsingDeprecationWarning: 'parseString'
   ↪  deprecated - use 'parse_string'
    parse = parser.parseString(pattern)

../../../../Library/Python/3.9/lib/python/site-packages/matplotlib/_fontconfi⌋
↪  g_pattern.py:89
../../../../Library/Python/3.9/lib/python/site-packages/matplotlib/_fontconfi⌋
↪  g_pattern.py:89
../../../../Library/Python/3.9/lib/python/site-packages/matplotlib/_fontconfi⌋
↪  g_pattern.py:89
```

```
../../../../Library/Python/3.9/lib/python/site-packages/matplotlib/_fontconfi
↪  g_pattern.py:89
../../../../Library/Python/3.9/lib/python/site-packages/matplotlib/_fontconfi
↪  g_pattern.py:89
../../../../Library/Python/3.9/lib/python/site-packages/matplotlib/_fontconfi
↪  g_pattern.py:89
  /Users/yoyontzin/Library/Python/3.9/lib/python/site-packages/matplotlib/_fo
   ↪  ntconfig_pattern.py:89: PyparsingDeprecationWarning: 'resetCache'
   ↪  deprecated - use 'reset_cache'
    parser.resetCache()

../../../../Library/Python/3.9/lib/python/site-packages/matplotlib/_mathtext.
↪  py:45
  /Users/yoyontzin/Library/Python/3.9/lib/python/site-packages/matplotlib/_ma
   ↪  thtext.py:45: PyparsingDeprecationWarning: 'enablePackrat' deprecated -
   ↪  use 'enable_packrat'
    ParserElement.enablePackrat()

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
3 passed, 14 warnings in 0.52s
```

11. Scripts globales y corridas de integración

11.1. Script maestro de validación

```
#!/usr/bin/env bash
set -euo pipefail

ROOT="$(cd "$(dirname "$0")" && pwd)"

echo "[1/4] Tests core"
cd "$ROOT/padic-grn-core"
python3 -m pytest -q

echo "[2/4] Tests search"
cd "$ROOT/padic-grn-search"
python3 -m pytest -q

echo "[3/4] Tests viz"
cd "$ROOT/padic-grn-viz"
python3 -m pytest -q

echo "[4/4] Smoke web backend import"
cd "$ROOT/web-demo-institucional/backend"
python3 -c "import app; print('web_backend_import_ok')"

echo "Validaciones completadas."
```

11.2. Evidencia de casos de estudio

```
# μ_A-maximizing orderings for A. thaliana (40 total, sorted by μ_E)
# Note: these are the 40 μ_A-optimal orderings (μ_A* = 54,560), NOT the 4
↪  μ_E-optimal orderings.
# For the μ_E-minimizers (μ_E* = 26,776), see ATHALIANA_OFFICIAL_OPTIMA.csv.
```

rank	μ_E	μ_A	μ_I	ordering
1	26976	54560	16768	12 3 4 8 1 5 7 2 10 6 11 9 0
2	26976	54560	16768	12 3 4 8 5 1 7 2 10 6 11 9 0
3	26976	54560	16768	12 4 3 8 1 5 7 2 10 6 11 9 0
4	26976	54560	16768	12 4 3 8 5 1 7 2 10 6 11 9 0
5	28000	54560	15744	3 4 8 1 5 7 2 10 6 12 11 9 0
6	28000	54560	15744	3 4 8 5 1 7 2 10 6 12 11 9 0
7	28000	54560	15744	4 3 8 1 5 7 2 10 6 12 11 9 0
8	28000	54560	15744	4 3 8 5 1 7 2 10 6 12 11 9 0
9	28384	54560	15360	3 4 8 1 5 7 2 10 12 6 11 9 0
10	28384	54560	15360	3 4 8 5 1 7 2 10 12 6 11 9 0
11	28384	54560	15360	4 3 8 1 5 7 2 10 12 6 11 9 0
12	28384	54560	15360	4 3 8 5 1 7 2 10 12 6 11 9 0
13	28448	54560	15296	3 4 8 1 5 7 2 12 10 6 11 9 0
14	28448	54560	15296	3 4 8 5 1 7 2 12 10 6 11 9 0
15	28448	54560	15296	4 3 8 1 5 7 2 12 10 6 11 9 0
16	28448	54560	15296	4 3 8 5 1 7 2 12 10 6 11 9 0
17	29024	54560	14720	3 4 8 1 5 12 7 2 10 6 11 9 0
18	29024	54560	14720	3 4 8 1 12 5 7 2 10 6 11 9 0
19	29024	54560	14720	3 4 8 5 1 12 7 2 10 6 11 9 0
20	29024	54560	14720	3 4 8 5 12 1 7 2 10 6 11 9 0
21	29024	54560	14720	3 4 8 12 1 5 7 2 10 6 11 9 0
22	29024	54560	14720	3 4 8 12 5 1 7 2 10 6 11 9 0
23	29024	54560	14720	4 3 8 1 5 12 7 2 10 6 11 9 0
24	29024	54560	14720	4 3 8 1 12 5 7 2 10 6 11 9 0
25	29024	54560	14720	4 3 8 5 1 12 7 2 10 6 11 9 0
26	29024	54560	14720	4 3 8 5 12 1 7 2 10 6 11 9 0
27	29024	54560	14720	4 3 8 12 1 5 7 2 10 6 11 9 0
28	29024	54560	14720	4 3 8 12 5 1 7 2 10 6 11 9 0
29	30048	54560	13696	3 4 8 1 5 7 12 2 10 6 11 9 0
30	30048	54560	13696	3 4 8 5 1 7 12 2 10 6 11 9 0
31	30048	54560	13696	4 3 8 1 5 7 12 2 10 6 11 9 0
32	30048	54560	13696	4 3 8 5 1 7 12 2 10 6 11 9 0
33	31072	54560	12672	3 4 12 8 1 5 7 2 10 6 11 9 0
34	31072	54560	12672	3 4 12 8 5 1 7 2 10 6 11 9 0
35	31072	54560	12672	4 3 12 8 1 5 7 2 10 6 11 9 0
36	31072	54560	12672	4 3 12 8 5 1 7 2 10 6 11 9 0
37	35168	54560	8576	3 12 4 8 1 5 7 2 10 6 11 9 0
38	35168	54560	8576	3 12 4 8 5 1 7 2 10 6 11 9 0
39	35168	54560	8576	4 12 3 8 1 5 7 2 10 6 11 9 0
40	35168	54560	8576	4 12 3 8 5 1 7 2 10 6 11 9 0

Recálculo de (μ_E, μ_A, μ_I) para órdenes citadas como óptimas

Método: `compute_scores` (`supplementary/code/core/compute_mu_generalized.py`) sobre los CSV actuales.

Aclaración técnica (*A. thaliana*): el mínimo **global** de μ_E es **26776** (cuatro ordenes; filas *ORDER_A oficial* y órbita). Las filas *JSON* corresponden a maximizadores de μ_A; ahí el menor μ_E es **26976**, no el mínimo global. El apéndice del paper y `VERIFICACION_SIMETRIAS_Y_NUMEROS.md` explican la distinción.

Red	Etiqueta	Orden (índices 0…N−1)	μ_E	μ_A	μ_I
Drosophila	π* B&B (eq. paper)	`0,5,3,4,2,1`	64	156	100
Drosophila	órbita μ_E mín (4/4) #1	`0,5,3,4,2,1`	64	156	100
Drosophila	órbita μ_E mín (4/4) #2	`0,5,4,3,2,1`	64	156	100
Drosophila	órbita μ_E mín (4/4) #3	`5,0,3,4,2,1`	64	156	100
Drosophila	órbita μ_E mín (4/4) #4	`5,0,4,3,2,1`	64	156	100
Drosophila	**μ_A máx global=156, 8 órdenes**	—	—	—
Drosophila	μ_A-max #1	`0,5,3,4,2,1`	64	156	100
Drosophila	μ_A-max #2	`0,5,4,3,2,1`	64	156	100
Drosophila	μ_A-max #3	`5,0,3,4,2,1`	64	156	100
Drosophila	μ_A-max #4	`5,0,4,3,2,1`	64	156	100
Drosophila	μ_A-max #5	`5,3,0,4,2,1`	96	156	68
Drosophila	μ_A-max #6	`5,3,4,0,2,1`	88	156	76
Drosophila	μ_A-max #7	`5,4,0,3,2,1`	96	156	68
Drosophila	μ_A-max #8	`5,4,3,0,2,1`	88	156	76
C. elegans	π* único μ_E (apéndice)	`1,0,2,3,6,5,4,7`	956	392	444
C. elegans	ejemplo μ_A-opt (apéndice)	`0,2,3,1,6,5,7,4`	1088	440	264
C. elegans	**μ_A máx global=440, 4 órdenes**	—	—	—
C. elegans	μ_A-max #1	`0,2,3,1,6,5,7,4`	1088	440	264
C. elegans	μ_A-max #2	`0,3,2,1,6,5,7,4`	1088	440	264
C. elegans	μ_A-max #3	`2,0,3,1,6,5,7,4`	1088	440	264
C. elegans	μ_A-max #4	`3,0,2,1,6,5,7,4`	1088	440	264
A. thaliana	ORDER_A oficial (μ_E global mín)	`12,3,4,8,5,1,7,2,10,11,9,6,0`	26776	53880	17648
A. thaliana	misma órbita μ_E #1	`12,3,4,8,1,5,7,2,10,11,9,6,0`	26776	53880	17648
A. thaliana	misma órbita μ_E #2	`12,3,4,8,5,1,7,2,10,11,9,6,0`	26776	53880	17648
A. thaliana	misma órbita μ_E #3	`12,4,3,8,1,5,7,2,10,11,9,6,0`	26776	53880	17648
A. thaliana	misma órbita μ_E #4	`12,4,3,8,5,1,7,2,10,11,9,6,0`	26776	53880	17648
A. thaliana	**40 órdenes μ_A-max (JSON), μ_A*=54560**	—	—	—
A. thaliana	JSON #1	`12,3,4,8,1,5,7,2,10,6,11,9,0`	26976	54560	16768
A. thaliana	JSON #2	`12,3,4,8,5,1,7,2,10,6,11,9,0`	26976	54560	16768
A. thaliana	JSON #3	`12,4,3,8,1,5,7,2,10,6,11,9,0`	26976	54560	16768
A. thaliana	JSON #4	`12,4,3,8,5,1,7,2,10,6,11,9,0`	26976	54560	16768
A. thaliana	JSON #5	`3,4,8,1,5,7,2,10,6,12,11,9,0`	28000	54560	15744
A. thaliana	JSON #6	`3,4,8,5,1,7,2,10,6,12,11,9,0`	28000	54560	15744
A. thaliana	JSON #7	`4,3,8,1,5,7,2,10,6,12,11,9,0`	28000	54560	15744
A. thaliana	JSON #8	`4,3,8,5,1,7,2,10,6,12,11,9,0`	28000	54560	15744
A. thaliana	JSON #9	`3,4,8,1,5,7,2,10,12,6,11,9,0`	28384	54560	15360
A. thaliana	JSON #10	`3,4,8,5,1,7,2,10,12,6,11,9,0`	28384	54560	15360

| A. thaliana | JSON #11 | `4,3,8,1,5,7,2,10,12,6,11,9,0` | 28384 | 54560 |
↪ 15360 |
| A. thaliana | JSON #12 | `4,3,8,5,1,7,2,10,12,6,11,9,0` | 28384 | 54560 |
↪ 15360 |
| A. thaliana | JSON #13 | `3,4,8,1,5,7,2,12,10,6,11,9,0` | 28448 | 54560 |
↪ 15296 |
| A. thaliana | JSON #14 | `3,4,8,5,1,7,2,12,10,6,11,9,0` | 28448 | 54560 |
↪ 15296 |
| A. thaliana | JSON #15 | `4,3,8,1,5,7,2,12,10,6,11,9,0` | 28448 | 54560 |
↪ 15296 |
| A. thaliana | JSON #16 | `4,3,8,5,1,7,2,12,10,6,11,9,0` | 28448 | 54560 |
↪ 15296 |
| A. thaliana | JSON #17 | `3,4,8,1,5,12,7,2,10,6,11,9,0` | 29024 | 54560 |
↪ 14720 |
| A. thaliana | JSON #18 | `3,4,8,1,12,5,7,2,10,6,11,9,0` | 29024 | 54560 |
↪ 14720 |
| A. thaliana | JSON #19 | `3,4,8,5,1,12,7,2,10,6,11,9,0` | 29024 | 54560 |
↪ 14720 |
| A. thaliana | JSON #20 | `3,4,8,5,12,1,7,2,10,6,11,9,0` | 29024 | 54560 |
↪ 14720 |
| A. thaliana | JSON #21 | `3,4,8,12,1,5,7,2,10,6,11,9,0` | 29024 | 54560 |
↪ 14720 |
| A. thaliana | JSON #22 | `3,4,8,12,5,1,7,2,10,6,11,9,0` | 29024 | 54560 |
↪ 14720 |
| A. thaliana | JSON #23 | `4,3,8,1,5,12,7,2,10,6,11,9,0` | 29024 | 54560 |
↪ 14720 |
| A. thaliana | JSON #24 | `4,3,8,1,12,5,7,2,10,6,11,9,0` | 29024 | 54560 |
↪ 14720 |
| A. thaliana | JSON #25 | `4,3,8,5,1,12,7,2,10,6,11,9,0` | 29024 | 54560 |
↪ 14720 |
| A. thaliana | JSON #26 | `4,3,8,5,12,1,7,2,10,6,11,9,0` | 29024 | 54560 |
↪ 14720 |
| A. thaliana | JSON #27 | `4,3,8,12,1,5,7,2,10,6,11,9,0` | 29024 | 54560 |
↪ 14720 |
| A. thaliana | JSON #28 | `4,3,8,12,5,1,7,2,10,6,11,9,0` | 29024 | 54560 |
↪ 14720 |
| A. thaliana | JSON #29 | `3,4,8,1,5,7,12,2,10,6,11,9,0` | 30048 | 54560 |
↪ 13696 |
| A. thaliana | JSON #30 | `3,4,8,5,1,7,12,2,10,6,11,9,0` | 30048 | 54560 |
↪ 13696 |
| A. thaliana | JSON #31 | `4,3,8,1,5,7,12,2,10,6,11,9,0` | 30048 | 54560 |
↪ 13696 |
| A. thaliana | JSON #32 | `4,3,8,5,1,7,12,2,10,6,11,9,0` | 30048 | 54560 |
↪ 13696 |
| A. thaliana | JSON #33 | `3,4,12,8,1,5,7,2,10,6,11,9,0` | 31072 | 54560 |
↪ 12672 |
| A. thaliana | JSON #34 | `3,4,12,8,5,1,7,2,10,6,11,9,0` | 31072 | 54560 |
↪ 12672 |
| A. thaliana | JSON #35 | `4,3,12,8,1,5,7,2,10,6,11,9,0` | 31072 | 54560 |
↪ 12672 |
| A. thaliana | JSON #36 | `4,3,12,8,5,1,7,2,10,6,11,9,0` | 31072 | 54560 |
↪ 12672 |
| A. thaliana | JSON #37 | `3,12,4,8,1,5,7,2,10,6,11,9,0` | 35168 | 54560 |
↪ 8576 |

```
| A. thaliana |   JSON #38 | `3,12,4,8,5,1,7,2,10,6,11,9,0` | 35168 | 54560 |
↪  8576 |
| A. thaliana |   JSON #39 | `4,12,3,8,1,5,7,2,10,6,11,9,0` | 35168 | 54560 |
↪  8576 |
| A. thaliana |   JSON #40 | `4,12,3,8,5,1,7,2,10,6,11,9,0` | 35168 | 54560 |
↪  8576 |
| A. thaliana | **10 representantes tab:muA (apéndice)** | — | — | — | — |
| A. thaliana |   órbita rep #1 | `12,3,4,8,1,5,7,2,10,6,11,9,0` | 26976 |
↪  54560 | 16768 |
| A. thaliana |   órbita rep #2 | `3,4,8,1,5,7,2,10,6,12,11,9,0` | 28000 |
↪  54560 | 15744 |
| A. thaliana |   órbita rep #3 | `3,4,8,1,5,7,2,10,12,6,11,9,0` | 28384 |
↪  54560 | 15360 |
| A. thaliana |   órbita rep #4 | `3,4,8,1,5,7,2,12,10,6,11,9,0` | 28448 |
↪  54560 | 15296 |
| A. thaliana |   órbita rep #5 | `3,4,8,1,5,12,7,2,10,6,11,9,0` | 29024 |
↪  54560 | 14720 |
| A. thaliana |   órbita rep #6 | `3,4,8,1,12,5,7,2,10,6,11,9,0` | 29024 |
↪  54560 | 14720 |
| A. thaliana |   órbita rep #7 | `3,4,8,12,1,5,7,2,10,6,11,9,0` | 29024 |
↪  54560 | 14720 |
| A. thaliana |   órbita rep #8 | `3,4,8,1,5,7,12,2,10,6,11,9,0` | 30048 |
↪  54560 | 13696 |
| A. thaliana |   órbita rep #9 | `3,4,12,8,1,5,7,2,10,6,11,9,0` | 31072 |
↪  54560 | 12672 |
| A. thaliana |   órbita rep #10 | `3,12,4,8,1,5,7,2,10,6,11,9,0` | 35168 |
↪  54560 | 8576 |

# Verificación: números, óptimos y simetrías

**Fuente de verdad numérica:** `compute_scores`
↪  (`supplementary/code/core/compute_mu_generalized.py`) sobre los CSV en
↪  `supplementary/data/`.
**Arabidopsis (40 órdenes μ_A-máximos):**
↪  `athaliana_bb_server/optima_closed_under_symmetry.json` +
↪  `athaliana/transitions.csv`.

---

## 1. *Drosophila melanogaster* (`transitions_drosophila.csv`)

| Comprobación | Resultado |
|--------------|-----------|
| Genes (bits) | 0=Bcd, 1=Hb, 2=Kr, 3=Kni, 4=Gt, 5=Cad (como en
↪  `drosophila_full_analysis.py`) |
| π* canónico | `[0,5,3,4,2,1]` = Bcd, Cad, Kni, Gt, Kr, Hb |
| Terna bajo π* | **(μ_E, μ_A, μ_I) = (64, 156, 100)** — coincide con
↪  `appendix_data_drosophila.tex` |
| Mínimo global μ_E (exhaustivo 720) | **64**, exactamente **4** órdenes |
| Simetría **en genes** | Swaps **(0↔5)** y **(3↔4)** generan un grupo $C_2
↪  \times C_2$ |
| Órbita desde `[0,5,3,4,2,1]` | **4** tuplas; **coincide exactamente** con el
↪  conjunto de minimizadores globales de μ_E |
| Invariante de scores | Las 4 tuplas tienen la misma terna (64, 156, 100) |
```

El texto del paper (“pares en posiciones 0–1 y 2–3”) se interpreta como **esos pares de genes** en las primeras cuatro posiciones del orden elegido; la implementación correcta es intercambiar **índices de genes** 0↔5 y 3↔4 dondequiera que estén.

---

2. *C. elegans* (`transitions_celegans.csv`)

Comprobación	Resultado
π* del apéndice	`[1,0,2,3,6,5,4,7]`
Terna bajo π*	**(956, 392, 444)** — coincide con `appendix_data_celegans.tex`
μ_E mínimo global (exhaustivo 8!)	**956**, **único** minimizador
μ_A máximo global	**440**, exactamente **4** órdenes
Intersección μ_E-ópt ∩ μ_A-ópt	**vacía** (0) — coincide con el apéndice

No se documenta un $C_2 \times C_2$ que deje invariante el funcional como en *Arabidopsis* / *Drosophila*; la estructura aquí es “un único mínimo de μ_E” vs “cuatro máximos de μ_A”.

---

3. *Arabidopsis thaliana* (`supplementary/data/athaliana/transitions.csv`)

Comprobación	Resultado
Simetría en **genes**	Transposiciones **(3↔4)** EMF1/LFY y **(5↔1)** AP2/FT
Mínimo **global** de μ_E (4 órdenes)	Terna **(26776, 53880, 17648)** — `ORDER_A` en `results/ATHALIANA_OFFICIAL_OPTIMA.csv`
40 órdenes μ_A-máximos (JSON)	μ_A = **54560** siempre; μ_E entre **26976** y **35168**
μ_E mínimo **restringido** a esas 40	**26976** (órbita 1); no es el mínimo global (26776)
Intersección {μ_E-global} ∩ {μ_A-global}	**Vacía** (los cuatro de μ_E tienen μ_A = 53880 < 54560)

Coherencia con el LaTeX

- `03_application_athaliana.tex` y el párrafo *Two-criteria analysis* de `appendix_data_athaliana.tex` (terna **26 776, 53 880, 17 648**, intersección vacía) coinciden con `compute_scores` sobre el CSV actual.
- La tabla `tab:muA_full_representatives` lista **solo** maximizadores de μ_A; la columna μ_E = **26 976** en la órbita 1 es el menor μ_E **sujeto a** μ_A = μ_A*, no el mínimo global. El pie de tabla en el `.tex` lo aclara.

---

4. Otros CSV (`transitions_emt.csv`, `transitions_celegans_colega.csv`, etc.)

- **EMT:** no hay sección principal en el Paper II v3.1; el CSV es auxiliar.

```
- **Colega vs Huang:** ver `NOTA_TABLA_COLEGA_VS_ARTICULO.md`; la tabla
↪  canónica del artículo es `transitions_celegans.csv` generada con
↪  `build_celegans_transitions_huang.py`.

---

## Resumen

| Red | Simetría documentada (genes) | Verificado |
|-----|------------------------------|------------|
| *Drosophila* | (0↔5), (3↔4) → 4 órdenes μ_E-mín | Sí |
| *A. thaliana* | (3↔4), (5↔1); 4 órbitas μ_E-global ≠ 40 órbitas μ_A-global |
↪  Sí; cuidado: 26 976 = min μ_E **entre** μ_A-máx, no global |
| *C. elegans* | No el mismo patrón $C_2\times C_2$ en el funcional (1 vs 4
↪  óptimos) | N/A; números del apéndice OK |
```

12. Applets y UI web reportada

12.1. Estado actual

El demo web institucional incluye frontend HTML, backend mínimo y guía de despliegue para visualización y pruebas funcionales.

12.2. Alcance operativo del demo web

La interfaz HTML institucional presentada en este expediente corresponde a la versión operativa utilizada para documentación y capturas del demo.

12.3. README del demo web

```
# Web Demo Institucional

Demo para integrar en sitio institucional y permitir pruebas controladas de
↪  los softwares:

- `padic-grn-core`
- `padic-grn-search`
- `padic-grn-viz`

## Estructura

- `frontend/` (UI)
- `backend/` (API)
- `docs/` (operación/despliegue)

## Estado

Versión operativa para visualización institucional y pruebas funcionales de la
suite.
```

12.4. Frontend HTML

```
<!doctype html>
<html lang="es">
```

```
  <head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <title>Demo p-adic GRN</title>
    <style>
      body { font-family: sans-serif; margin: 2rem; max-width: 900px; }
      code { background: #f3f3f3; padding: 0.1rem 0.3rem; }
      .box { border: 1px solid #ddd; border-radius: 8px; padding: 1rem;
      ↪  margin-top: 1rem; }
    </style>
  </head>
  <body>
    <h1>Demo institucional — p-adic GRN Suite</h1>
    <p>Estado inicial de la interfaz de demostración.</p>
    <div class="box">
      <h2>Servicios</h2>
      <ul>
        <li><code>padic-grn-core</code> (métricas)</li>
        <li><code>padic-grn-search</code> (GA/B&amp;amp;B)</li>
        <li><code>padic-grn-viz</code> (visualización)</li>
      </ul>
      <p>Backend health endpoint esperado: <code>/health</code>.</p>
    </div>
  </body>
  </html>
```

12.5. Backend

```
"""API mínima para demo institucional."""

from __future__ import annotations

from http.server import BaseHTTPRequestHandler, HTTPServer
import json


class Handler(BaseHTTPRequestHandler):
    def do_GET(self) -> None:  # noqa: N802
        if self.path == "/health":
            payload = {"status": "ok", "service": "web-demo-backend"}
            body = json.dumps(payload).encode("utf-8")
            self.send_response(200)
            self.send_header("Content-Type", "application/json;
            ↪  charset=utf-8")
            self.send_header("Content-Length", str(len(body)))
            self.end_headers()
            self.wfile.write(body)
            return
        self.send_response(404)
        self.end_headers()


def main() -> None:
    server = HTTPServer(("127.0.0.1", 8080), Handler)
    print("Backend demo escuchando en http://127.0.0.1:8080")
```

```
    server.serve_forever()


if __name__ == "__main__":
    main()
```

12.6. Guía de despliegue

```
# Despliegue básico

## Backend

```bash
cd software-suite/web-demo-institucional/backend
python3 app.py
```

Frontend estático

Servir `frontend/index.html` con cualquier servidor estático institucional.

Componentes del despliegue

1. API backend para verificación (`/health`).
2. Interfaz frontend estática para demostración.
3. Integración con módulos de cálculo y reportes de la suite.
```

12.7. Capturas de pantalla del demo

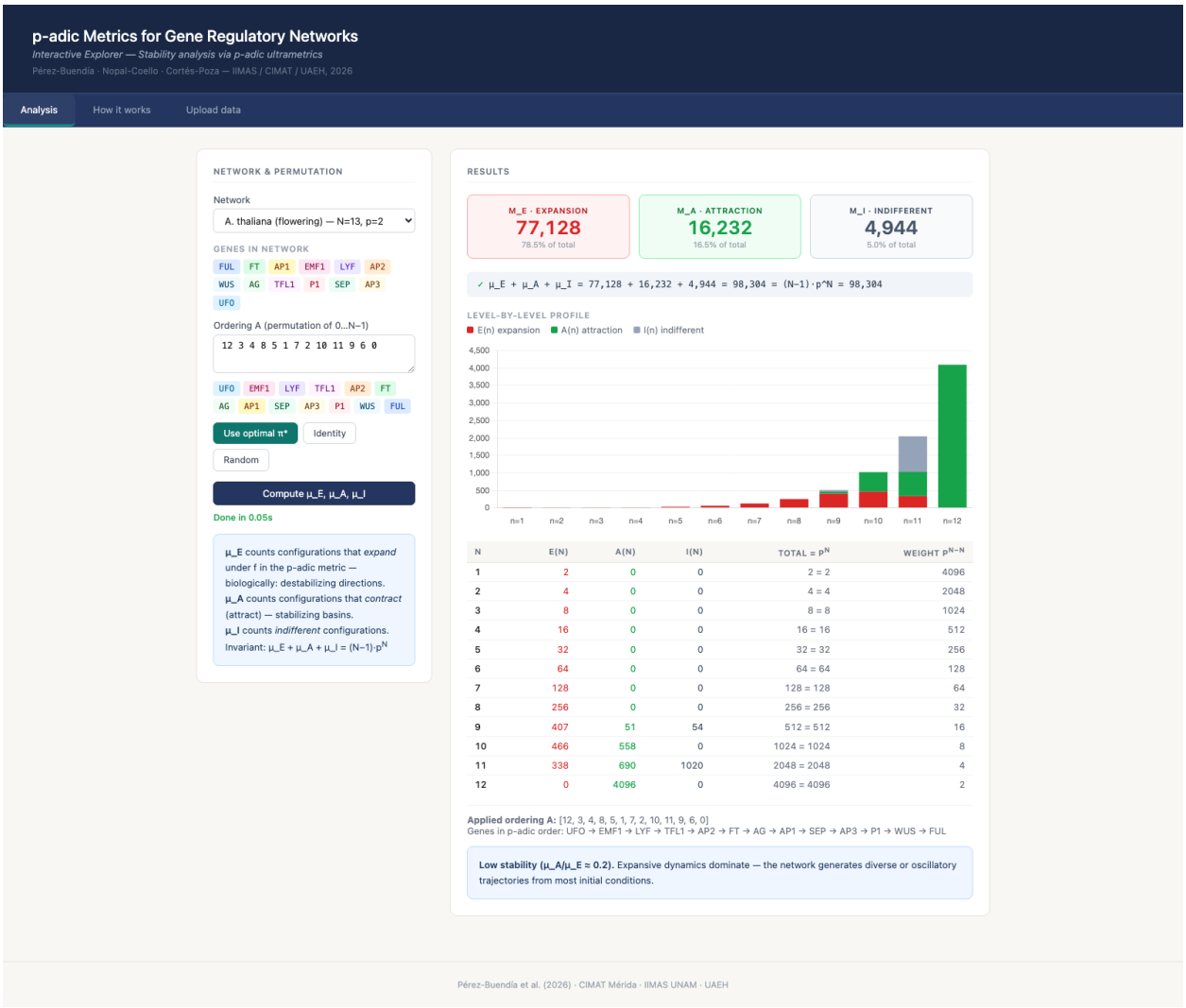


Figura 1: Demo `padic-grn-demo.html` en ejecución real (escritorio): cálculo de `mu_E`, `mu_A`, `mu_I`, gráfica de perfil y tabla de resultados.

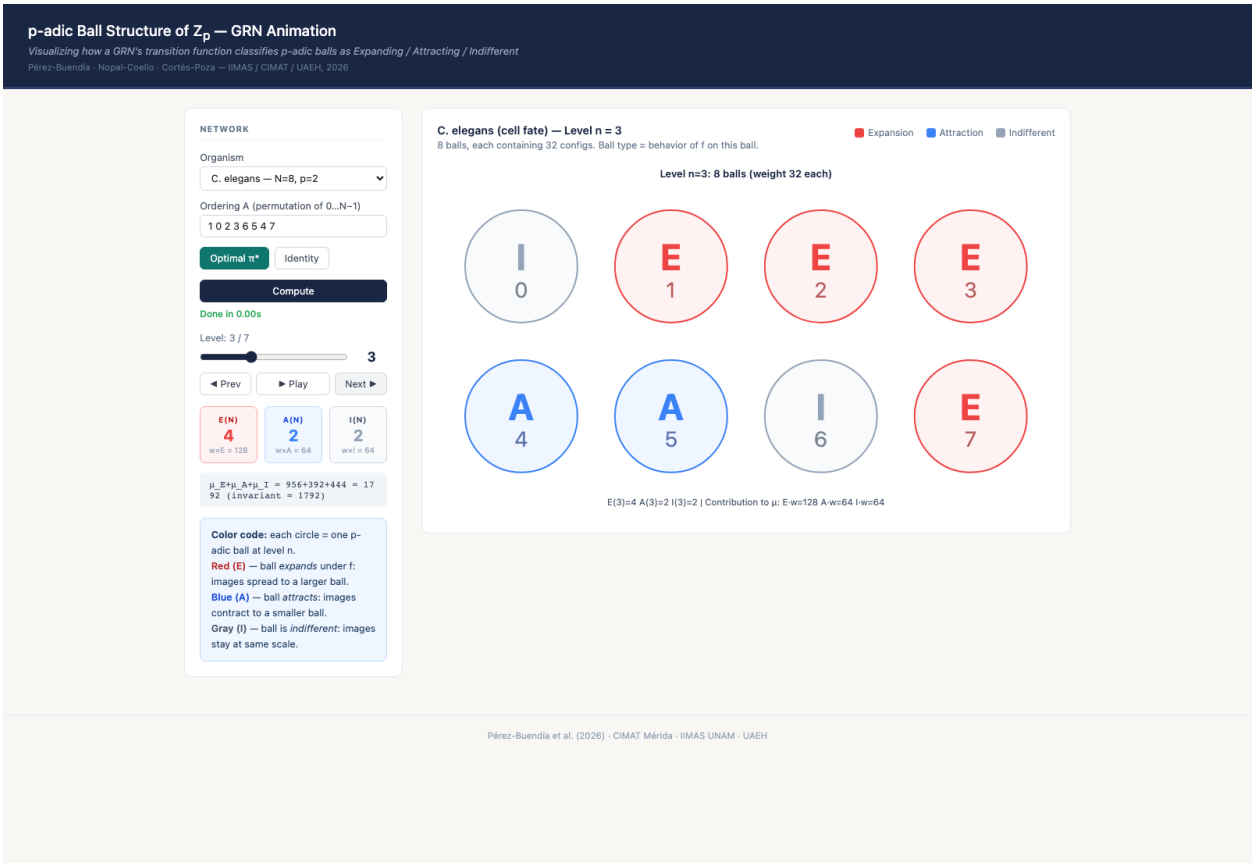


Figura 2: Demo `padic-balls-animation.html` en ejecución real (escritorio): clasificación de bolas p-ádicas y navegación por niveles.

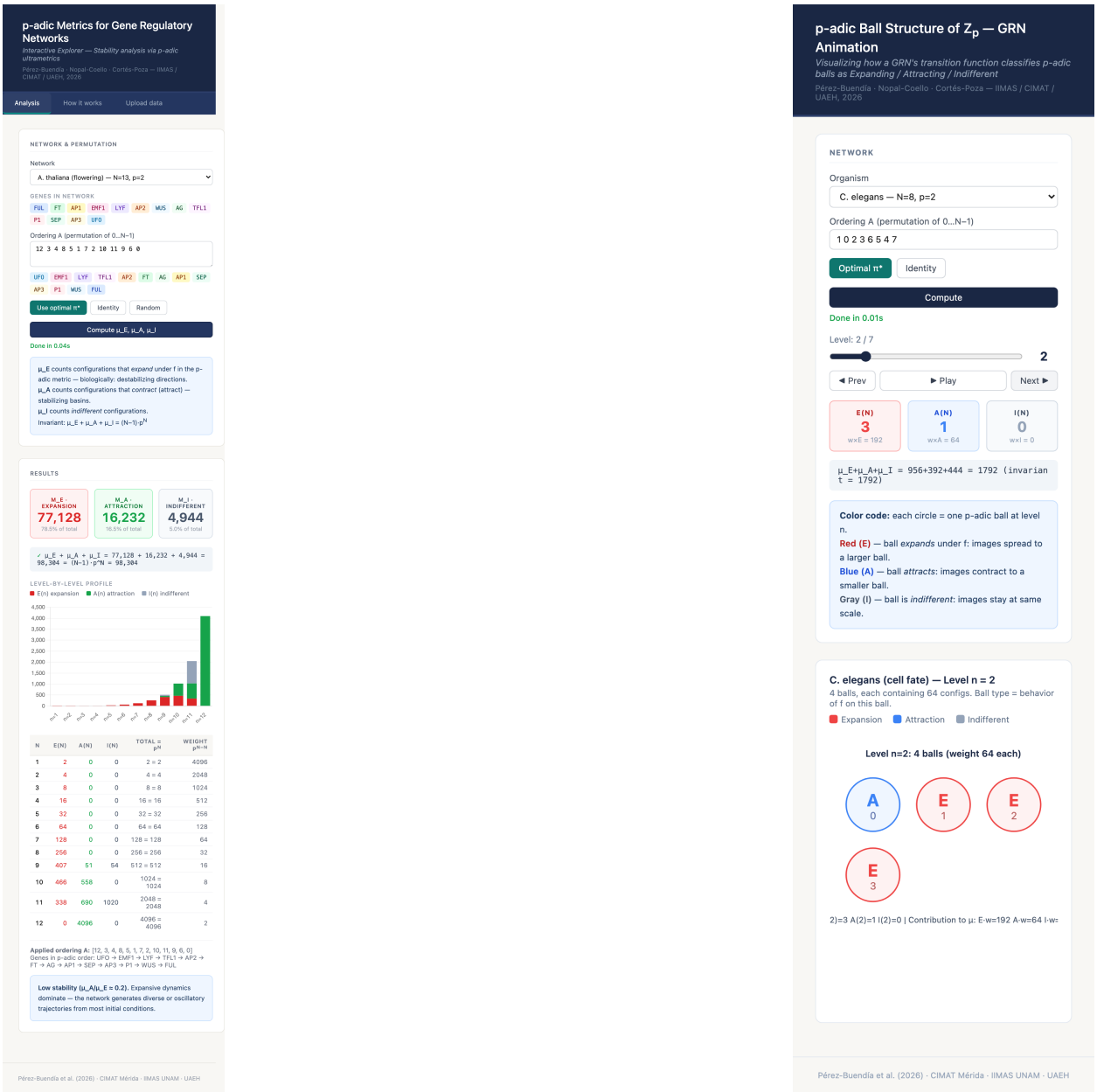


Figura 3: Ejecución móvil de los dos applets de análisis p-ádico con resultados y visualización activa.



Figura 4: Vista de portada del frontend institucional del demo web reportado.